PCI-SIG ENGINEERING CHANGE NOTICE

TITLE:	TEE Device Interface Security Protocol (TDISP)
DATE:	Introduced: 1 Dec 2020 Revised: 27 July 2022
AFFECTED DOCUMENT:	PCIe Base Specification Rev 5.0 + IDE ECN, 6.0
SPONSOR:	Advanced Micro Devices, ARM Ltd., Intel Corporation, Microsoft

Part I

1. Summary of the Functional Changes

Virtualization-based Trusted Execution Environments (TEEs) are used to host confidential computing workloads that may be isolated from hosting environments, i.e., the VMM is not required to be trusted. Hereafter, such TEEs are referred to as Trusted Execution Environment VMs (TVMs) to distinguish them from traditional virtual machines. When a portion of a device (a "TEE Device Interface" (TDI) - the unit of direct assignment, e.g., a Virtual Function when using SR-IOV) is assigned to a TVM, it is necessary to establish and maintain a trusted execution environment for the composition.

This document defines the TEE Device Interface Security Protocol (TDISP) - An architecture for trusted I/O virtualization providing the following functions:

1. Establishing a trust relationship between a TVM and a device.
2. Securing the interconnect between the host and device.
3. Attach and detach a TDI to a TVM in a trusted manner.

Builds upon the foundation provided by CMA/SPDM and IDE.

2. Benefits as a Result of the Changes

Systems will be able to establish and maintain composed TEEs including one or more devices, providing the ability to efficiently secure workloads in a virtualized system environment.

3. Assessment of the Impact

New hardware and software will be required to implement and operate this optional capability.

4. Analysis of the Hardware Implications

Defines a data object protocol and associated mechanisms. In addition to these mechanisms, hardware running secured workloads is required to be implemented securely, and to operate in a secure way.

5. Analysis of the Software Implications

New software will be required to implement this capability.

6. Analysis of the C&I Test Implications

C&I Tests need to be updated to handle newly defined fields in existing registers.

Part II

Detailed Description of the change

Add to Terms and Acronyms:

TEE Device Interface (TDI)

The unit of assignment for an IO-virtualization capable device. For example, a TDI may be an entire Device, a non-IOV Function, a PF (and possibly its subordinate VFs), or a VF.

…

TEE-I/O

A conceptual framework for establishing and managing Trusted Execution Environments (TEEs) that include a composition of resources from one or more devices (see Chapter 11 – TEE Device Interface Security Protocol (TDISP)).

…

Trusted Computing Base (TCB)

A combination of hardware, firmware, and software, responsible for enforcing a security policy. Bugs or vulnerabilities occurring inside the TCB might jeopardize the security properties of the entire system. By contrast, parts of a computer system outside the TCB shall not be able to create a condition that would allow any more privileges than are granted to them in accordance with the security policy.

…

TVM

A Trusted Execution Environment Virtual Machine as defined in the TEE Device Interface Security Protocol (TDISP) reference architecture (see Chapter 11).

In Reference Documents, edit:

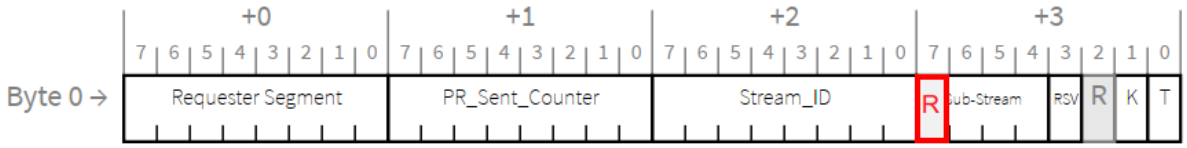
SPDM

DMTF Security Protocol & Data Model (SPDM) Specification - https://www.dmtf.org/dsp/DSP0274. IDE requires version 1.1 or above, TDISP requires version 1.2 or above.

Edit as shown:

2.2.1.2 Common Packet Header Fields for Flit Mode

…



§

Figure 2-13 ***OHC-C***

…

- For TLPs with OHC-C that are not IDE TLPs, the Sub-Stream[~~3~~2:0] field must be ~~0~~111b, and the Stream ID, PR_Sent_Counter, K and T fields/bits are Reserved.
- In earlier versions of this specification, Sub-Stream was 4 bits in Symbol 3, bits 7:4. Bit 7 is now Reserved. If TEE-IO Supported is Set, components must implement bit 7 as Reserved. If TEE-IO Supported is Clear, components are permitted to treat bit 7 as part of Sub-Stream

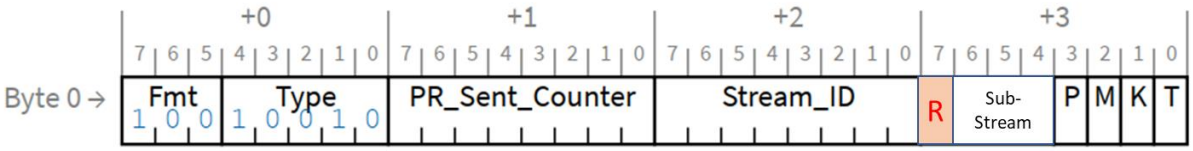
In Section 6.33, in the Implementation Note: Threat Model and Related Considerations for IDE, edit as shown:

… The details of how such TEEs are implemented and managed are outside the scope of IDE. The T bit is used for TEE management mechanisms (see Chapter 11 TDISP) ~~provides a mechanism to distinguish TLPs that are associated with a TEE~~. IDE mechanisms ensure that the T bit (like other TLP content) is secured during transit.

Edit as shown:

6.33.4 IDE TLPs

…



The IDE Prefix includes:

- …
- T bit – When Set, indicates the TLP originated from within a trusted execution environment (see § Section 6.33.1).
  - If the TEE-IO Supported bit in the Device Capabilities Register is Clear:
    - It is permitted for IDE TLPs to originate from both trusted and non-trusted execution environments, and the value of the T bit does not modify the handling of TLPs within IDE per se; the rules for trusted execution environments are not defined in this document.
    - The T bit must be Clear unless the use of the T bit has been explicitly defined by TEE management mechanisms outside the scope of this document.
  - If the TEE-IO Supported bit in the Device Capabilities Register is Set, this bit must be used for TEE management mechanisms as defined in Chapter 11 TDISP.

…

For each Sub-Stream there is a Sub-Stream identifier:

- ~~0~~000b – Posted Requests
- ~~0~~001b – Non-Posted Requests
- ~~0~~010b – Completions
- Values ~~0~~011b-~~0~~110b are Reserved
- ~~0~~111b - In NFM, Reserved; In FM, indicates a TLP that includes OHC-C but is not an IDE TLP.
- In earlier versions of this specification, Sub-Stream was 4 bits in Symbol 3, bits 7:4. Bit 7 is now Reserved. If the TEE-IO Supported bit in the Device Capabilities Register is Set, components must implement bit 7 as Reserved. If TEE-IO Supported is Clear, components are permitted to treat bit 7 as part of Sub-Stream
- ~~Values 1000b-11111 are permitted to be used for other uses not defined by this specification.~~

…

- For a TLP to be associated with a specific Selective IDE Stream, the following criteria must be satisfied:
  - …
  - For a Completion, the Stream ID ~~and T bit values in the Selective IDE Stream~~ must match those in the corresponding Non-Posted Request.
  - …
  - The TLP is selected or precluded through implementation-specific means. For example, the Transmitter could associate all Memory Requests initiated by one or more internal

Functions with a specific Selective IDE Stream, particularly when it is known that the Partner Port is a Root Port, and that all Requests initiated by that internal Function target system memory.

- If a TLP does not meet all applicable criteria above for a specific Selective IDE Stream, or if a TLP is precluded through implementation-specific means, the TLP must not be associated with any Selective IDE Stream.
- If the TEE-Limited Stream bit in the Selective IDE Stream Control Register is Set, only TLPs with the T bit Set are permitted to be associated with this Selective IDE Stream.
  - TLPs that otherwise would be associated with this Selective IDE Stream and have the T bit Clear must be Transmitted as non-IDE TLPs if Link IDE is not enabled, or as Link IDE TLPs if Link IDE is enabled.
- If supported, the TEE-Limited Stream bit must be configured in both Partner Ports before enabling a Selective IDE Stream.
  - It is permitted for the Partner Ports to have different values for the TEE-Limited Stream bit.

…

- Software must configure Selective IDE such that all Links on the entire path between the Partner Ports are operating either in FM, or in NFM.
- If **TEE-IO Supported** is Set, for an Endpoint Upstream Port:
  - A Request received on a Selective IDE Stream other than the Default Stream must be handled as an Unsupported Request if the Requester ID field of the Request is less than the RID Base or greater than the RID Limit in the Selective IDE RID Association Register block of the Stream.
    - In Flit Mode, if **Segment Captured** is Set*[footnote: The value of Segment Captured indicates that Segment values are being applied – the captured Segment value is not used for the tests defined here]*, the Request must include OHC-C with the Requester Segment Valid (RSV) bit Set, and the the Requester Segment value must also match the value of **Segment Base** in the Selective IDE RID Association Register block.
  - A Completion received on a Selective IDE Stream other than the Default Stream must be handled as an Unexpected Completion if the Completer ID field of the Completion is less than the RID Base or greater than the RID Limit in the Selective IDE RID Association Register block of the Stream.
    - In Flit Mode, if **Segment Captured** is Set, the Completion must include OHC-A5, and the Completer Segment value must also match the value of **Segment Base** in the Selective IDE RID Association Register block.
- If **TEE-IO Supported** is Set, for a Root Port:
  - A Request received on a Selective IDE Stream for which the Root Port is an IDE Terminus must be handled as an Unsupported Request if the Requester ID field of the Request is less than the RID Base or greater than the RID Limit in the Selective IDE RID Association Register block of the Stream.
    - In Flit Mode, if the Requester Segment Valid (RSV) bit is Set, the Requester Segment value must also match the value of **Segment Base** in the Selective IDE RID Association Register block.
  - A Completion received on a Selective IDE Stream for which the Root Port is an IDE Terminus must be handled as an Unexpected Completion if the Completer ID field of

the Completion is less than the RID Base or greater than the RID Limit in the Selective IDE RID Association Register block of the Stream.

- In Flit Mode, if the Completion includes OHC-A5, the Completer Segment value must also match the value of Segment Base in the Selective IDE RID Association Register block.

The use of Multicast (see § Section 6.14) with Selective IDE Streams is outside the scope of this specification.

Edit as shown:

6.33.8 Other IDE Rules

Rule for Non-Posted IDE Requests:

- If the TEE-IO Supported bit in the Device Capabilities Register is Clear, Requesters of Non-Posted Requests must check that the corresponding received Completion(s) be returned using the same Stream ID and the same T bit value as was associated with the Non-Posted Request – a violation is an IDE Check Failed error.
  - The Requester must transition to Insecure for the associated IDE Stream.
- If the TEE-IO Supported bit in the Device Capabilities Register is Set, Requesters of Non-Posted Requests must check that the corresponding received Completion(s) be returned using the same Stream ID as was associated with the Non-Posted Request – a violation is an IDE Check Failed error.
  - The Requester must transition to Insecure for the associated IDE Stream.

Edit as shown:

7.9.26.2 IDE Capability Register (Offset 04h)

…

Bit Location	Register Description	Attributes
…		
24	**TEE-Limited Stream Supported** – When Set, indicates that the TEE-Limited Stream control mechanism is supported. If **Selective IDE Streams Supported** is Clear, this bit is Reserved.	HwInit / RsvdP

7.9.26.5.2: Selective IDE Stream Control Register

…

Bit Location	Register Description	Attributes
…		
23	**TEE-Limited Stream** – When Set, requires that, for Requests, only those that have the T bit Set are permitted to be associated with this Stream. Must be configured while Selective IDE Stream Enable is Clear, during which time this bit is RW. When Selective IDE Stream Enable is Set set to 1, the setting is sampled, and this field bit becomes RO with reads returning the sampled value, during the time when Selective IDE Stream Enable remains Set. Reserved if **TEE-Limited Stream Supported** is Clear. Default value is 0b.	RW / RO / RsvdP

Edit as shown:

7.5.3.3 Device Capabilities Register (Offset 04h)

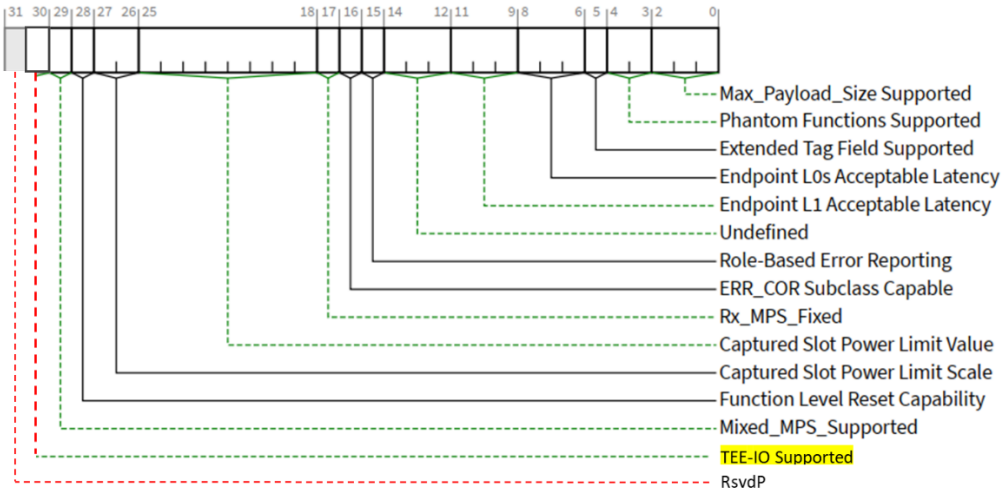
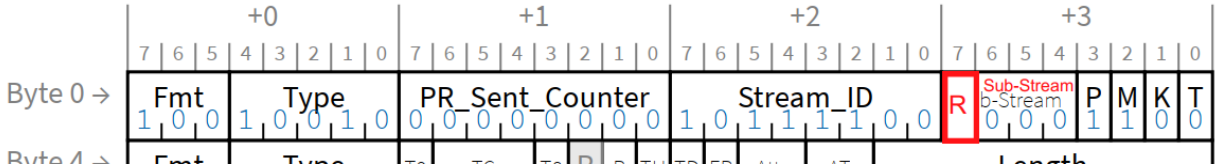


Figure 7-24 Device Capabilities Register

Bit Location	Register Description	Attributes
…		
30	**TEE-IO Supported** – When Set, this bit indicates that the Function implements the TEE-IO functionality as described by the TEE Device Interface Security Protocol (TDISP). See Chapter 11.	HwInit

Edit as shown, Appendix L: Example IDE TLP, figure L-2:


+0
+1
+2
+3
7 6 5 4 3 2 1 0
Byte 0 →
Fmt
Type
PR_Sent_Counter
Stream_ID
R
Sub-Stream
b-Stream
P
M
K
T
1 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 1 0 1 1 1 1 0 0 0 0 0 1 1 0 0

…

Add Chapter as shown:

11. TEE Device Interface Security Protocol (TDISP)

Trusted Execution Environments (TEEs) that include a composition of resources from one or more devices and the host require mechanisms to establish and manage trust relationships. Here we will use the term TEE-I/O to refer to a conceptual framework for performing such operations. This chapter defines a specific architecture for hosts and devices to participate in TEE-I/O (see Figure 11-1).

TEE-I/O builds upon existing capabilities for the direct assignment of devices to VMs, such as SR-IOV (Chapter 9) and ATS (Chapter 10), to establish Trusted Execution Environment VMs (TVMs). All VMs that are not TVMs are referred to as legacy VMs. In TEE-I/O, the VMM itself may not be trusted by TVMs, and mechanisms are provided to enable the TVM to make trust decisions based on the underlying hardware it is using. Although the VMM is not required to be trusted by TVMs, it continues to perform the resource allocation and system management functions as it does in non-TEE-I/O use models, but in such a way that the results can be tested. The VMM can be blocked from bypassing the security of the affected TVM(s). Legacy VMs that implicitly trust the VMM may co-exist with TVMs in a system.

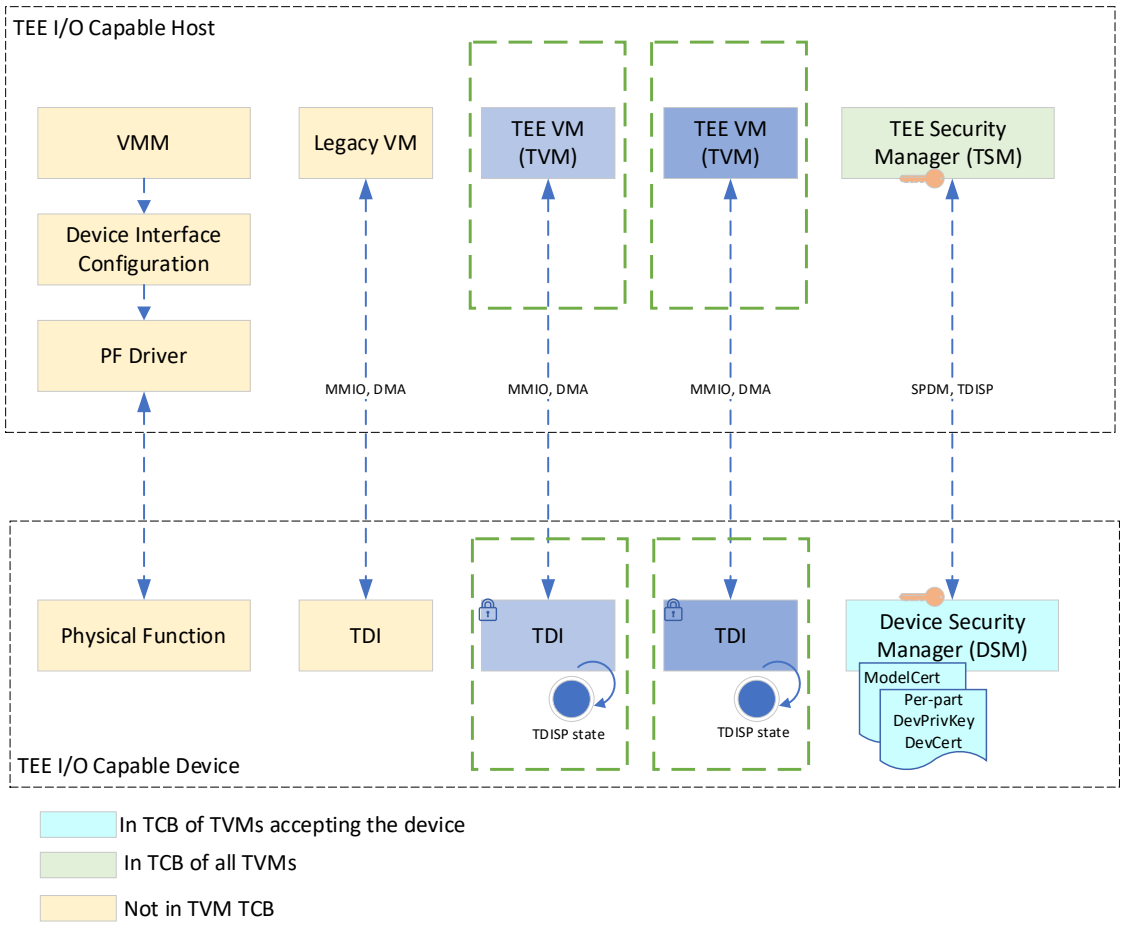


Figure 11-1 Conceptual View with Example Host and Device and Logical Communication Paths

The TEE security manager (TSM) is a logical entity in a host that is in the TCB for a TVM and enforces security policies on the host. The Device Security Manager (DSM) is a logical entity in the device that may be admitted into the TCB for a TVM by the TSM and enforces security policies on the device.

The TEE Device Interface (TDI) Security Protocol (TDISP) defines an architecture for devices that support TEE-I/O virtualization, providing the following functions:

1. Establishing a trust relationship between a TVM and a device.
2. Securing the interconnect between the host and device.
3. Attach and detach a TDI to a TVM in a trusted manner.

Although TDISP has been defined in relation to TEE-I/O as describe above, TDISP stands alone as a specification for devices, and such devices may be operated in systems using security architectures other than TEE-I/O, provided that the host functions required by TDISP are supported in an appropriate way by the system.

TDISP defines requirements for TDIs specifically, and also for the entire Device implementing TDIs, where in a specific instance, a TDI may be an entire Device, a non-IOV Function, a PF (and possibly its subordinate VFs), or a VF. Although it is permitted (and generally expected) that TDIs will be implemented such that they can be assigned to Legacy VMs, such use is not the focus of TDISP.

Implementation Note: Future TEE Extensions

TEE extensions for sub-function I/O-virtualization techniques will be covered in a future revision of this specification.

11.1. Overview of the TEE-I/O Security Model as it Relates to Devices

The TEE-I/O security model is primarily intended to apply to systems using device resources directly assigned to VMs, and this chapter generally assumes this use case. However, devices that are compliant to TDISP can potentially be used in other ways, and such use is not prohibited, although it is outside the scope of this specification.

The TEE-I/O considers all resources, including memory of all types, host processors, TDI's and, in some cases internal state, to be in one of two classes:

- "TEE-assignable" resources have the required trust/security capabilities to be assigned to a TEE. Once assigned to a TEE, these resources become "TEE-Owned."
- "Non-TEE-assignable" resources either do not possess the required trust/security capabilities to be assigned to a TEE or have been excluded by some implementation-specific mechanism. These resources may, and often do, have a critical role in system function, and therefore it is often desirable to appropriately secure them, although how this is done is outside the scope of this specification.

The TEE-I/O security model does not require the VMM to be trusted by TVMs. Therefore, devices supporting hardware-assisted I/O virtualization (e.g., SR-IOV) require security extensions to ensure that the device's virtualization model does not allow or require intervention by software outside the TVM trust boundary to perform operations that affect the confidentiality and/or integrity of TVM data in-flight or at-rest in the device. The primary focus of TDISP is to define the requirements for TDISP-compliant devices and the necessary elements outside of such devices required to support the TDISP architecture. Additional system capabilities required are outside the scope of TDISP.

TVM data, code, and execution state stored in an assigned device must be protected against:

- Confidentiality breaches: read access by entities (firmware, software, or hardware) not in the TCB of the TVM (such as other TVMs, VMM, etc.).

- Integrity: modification by entities (firmware, software, or hardware) not in the TCB of the TVM (such as other TVMs, VMM, etc.).

This security model does not require protection of TVMs against denial-of-service attacks in general. However, systems may impose a requirement that a TVM not have the ability to cause denial of service to other TVMs, VMM, or other VMs executing on the platform. The TSM by itself may not have all the capabilities needed to defend the platform against denial of service. Enforcing this property is the collective responsibility of the TSM, VMM, device and DSM, and is outside the scope of this specification.

The hardware assisted I/O virtualization schemes for direct I/O from TVMs to devices must address the following to preserve the confidentiality and integrity of the TVMs and the data moved between the TVMs and devices:

1. Authenticating device identity and measurement reporting - Device identities like Vendor ID and Device ID may be spoofed with malicious intent. Firmware executing on devices may have security vulnerabilities, or may have been tampered with. Device debug interfaces may be used to obtain low level access to the device hardware and thereby influence the security property of devices. The TVM must be able to cryptographically check the identity of the device, identity of the firmware components running on the device, and security state of the device (e.g., debug active). CMA/SPDM is used to support these requirements.
2. Device to Host communication Security - Physical access may be used to tamper with the data transferred between the host and the device. Transfers must be cryptographically protected to provide confidentiality, integrity, and replay protection to TVM data, and such schemes must also guard against violations of producer-consumer ordering. IDE is used to support these requirements. In some cases, e.g. for an RCiEP, it may be possible to ensure by construction that communication is not be susceptible to tampering, and therefore may not require the use of IDE. The TSM and DSM are both responsible for ensuring that Device/Host (and, when peer-to-peer is used, Device/Device) communication is secured by IDE, or by other means that satisfy use model requirements.
3. TEE Device Interface (TDI) management - DMA and interrupt remapping tables set up by the VMM may be tampered with by the VMM. The VMM administration of these tables (e.g., IOMMU TLB management, Device TLB management, Page Request handling, etc.) may additionally be tampered with by the VMM to influence the security of the TVM interaction with the device. The device must support locking down configurations of the TDI, reporting the configurations in a trusted manner, securely placing the TDIs into operational state, and subsequently tearing them down when the TDI is detached from a TVM. This chapter defines the mechanisms used to manage the security states of TDIs.
4. Device Security Architecture - Administrative interfaces (e.g., a PF) may be used to influence the security properties of the TDI used by the TVM. The device's security architecture must provide isolation and access control for TVM data in the device for protection against entities that are not in the trust boundary of the TVM. This chapter defines some device security architecture requirements, but additional requirements may exist for specific implementations that are outside the scope of this specification.

This chapter defines the wire protocol and the security objectives that are required to be implemented by the host and the device to be compatible with the TEE-I/O framework and the capabilities that need to be implemented to achieve specified security objectives. The implementation of such capabilities and the physical manifestation of the logical entities are outside the scope of this specification.

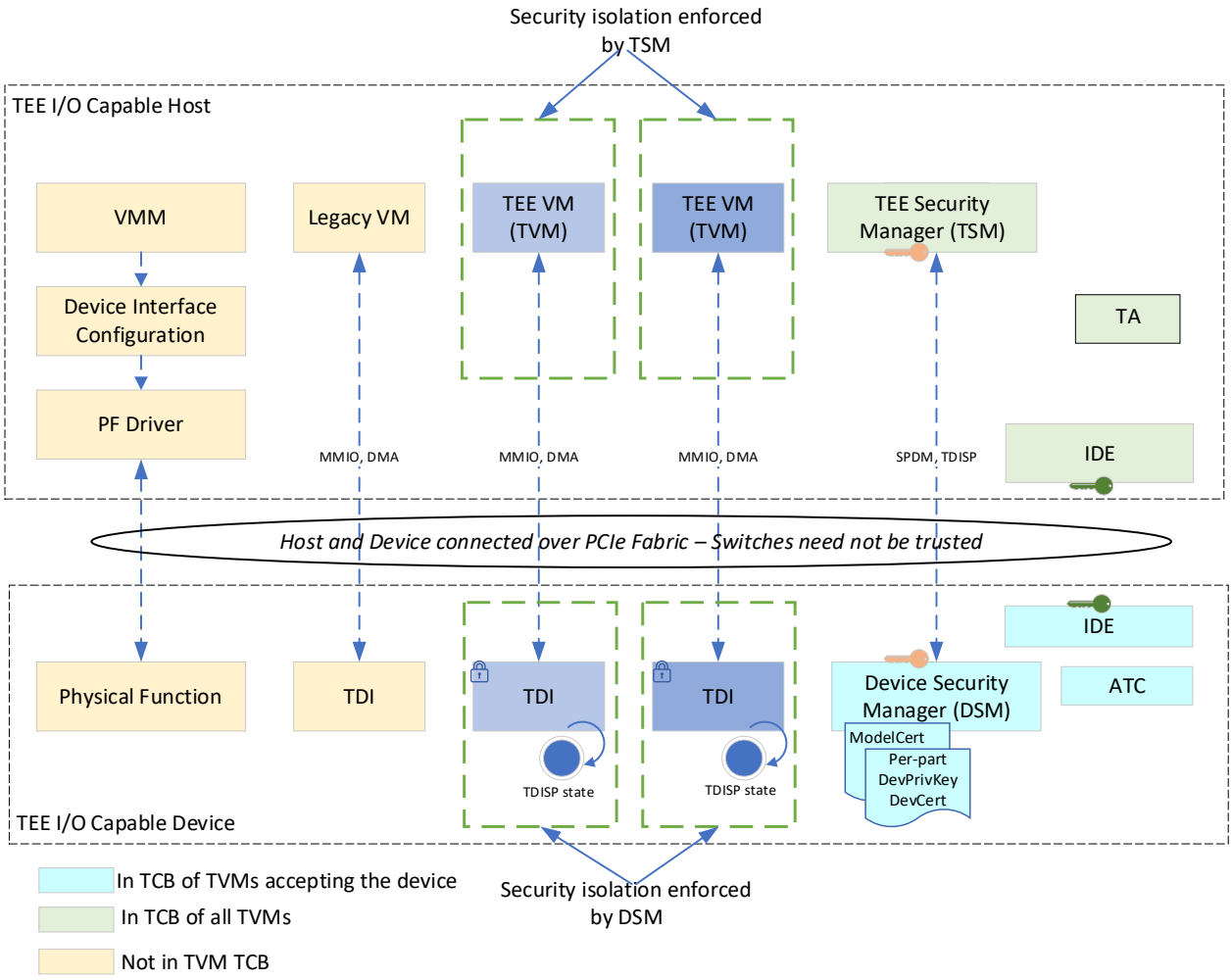


Figure 11-2 TDISP Host/Device Reference Architecture

Figure 11-2 illustrates key elements of the TDISP reference architecture. Typically, a PF is the resource management entity for a TDI and is managed by the PF driver in the VMM. The VMM and the PF driver are not required to be in the TCB of the TVMs. The VMM uses the PF to configure TDIs for assignment to TVMs. TEE-I/O requires the device to organize its hardware/software interfaces such that the PF cannot be used to affect the security of a TDI when it is in use by a TVM. The device must support mechanisms to lockdown the configurations of the TDI, when requested by the TSM, such that any modifications to the TDI configurations, once the TVM has accepted and started using the TDI, are detected as malicious actions. The device is required to implement a security architecture that protects the confidentiality and integrity of TVM data from being tampered with by the PF or other TDIs assigned to other TVMs or VMs. To ensure that error conditions can be appropriately managed, the device should implement Advanced Error Reporting (AER).

There are a variety of additional elements of the reference architecture. Software running on a Host CPU must be associated with a TEE via implementation-specific means. Memory can be a system-level resource or associated with a TDI, and is defined as either TEE memory or non-TEE memory. TEE memory must have mechanisms to ensure the confidentiality of TVM data, and may additionally provide integrity properties on the TVM data. Non-TEE memory is not assumed to have any such mechanisms.

System configuration of Memory Address routing mechanisms must be managed so as to ensure correct system operation, as misrouting of TLPs will in many cases result in conditions indistinguishable from an attack, in turn resulting in an error condition. Except when a peer-to-peer connection has been established between two TDIs, all Requests must be routed to the Root Complex, and in some cases this result is achieved by means of Access Control Services (see Section 6.12 Access Control Services (ACS)) mechanisms that modify the routing of TLPs.

When Links that could be subject to physical attacks are used, Integrity and Data Encryption (IDE) must be supported and enabled. The use of Selective IDE Streams minimizes the TCB and attack surface by allowing intermediate Switches to be excluded from the TCB. For Endpoint Upstream Ports connected directly to Root Ports, Link IDE meets the stated requirement of minimizing TCB and attack surface, and it is acceptable in such configurations to use Link IDE instead of Selective IDE, provided the TSM and DSM are able to provide acceptable security in this configuration. It is permitted for IDE streams established by the TSM to be used to carry TLPs associated with legacy VMs. Between the same two Ports, separate IDE streams per TDI do not provide additional protection against an adversary employing physical attacks on a Link, so only a single IDE Stream is required. The TLPs once decrypted and authenticated at the device or at the Root Port are in cleartext, and access control mechanisms put in place by the TSM on the host, and the DSM on the device, must provide confidentiality and integrity to the TLP contents against entities not in the TCB of the TVMs. How this is done is outside the scope of this specification.

An RCiEP or other TDI integrated into a Root Complex may not require use of IDE to protect the TLPs, and as such is not required to implement IDE. Such devices may use a Root Complex specific indication that is equivalent to the T bit in the IDE Prefix (NFM) /OHC-C (FM) to indicate that the TLP is associated with a TVM. For simplicity, such Root Complex specific indications are also referred to as the T bit, although this does not imply an implementation requirement. IDE Stream-specific checks and actions defined in later sections are not required for RCiEPs that do not implement IDE. An RCiEP may not require the use of [Secured SPDM] for protecting the communication between the TSM and DSM if it is possible to ensure the security of communication by other means.

In general, it is not assumed that system configuration relevant to TEE-I/O operation can be protected against inappropriate modification, and in some cases it may not even be possible to do so. Instead, system elements used with TEE-I/O, including TDISP-compliant devices, detect inappropriate modifications when it is possible to do so, and further protect themselves against security policy violations without depending on other elements of the system for assistance, for example by detecting non-IDE TLPs in cases where IDE is required, and by checking memory addresses in received Requests. The detection by one system element of an error condition results in that element entering a “fail safe” error state, but it may not in all cases be possible for this to be directly communicated to other system elements, for example because an attacker may block attempted notifications, and so in such cases error conditions must be inferred, for example through a TDI’s lack of responsiveness.

This chapter specifies the protocol used by the TSM and DSM to associate an IDE Stream ID to be used by the TDI. The T bit allows the originator of a TLP to indicate that a TLP is associated with a TVM. The T bit is used by the device and the host translation agent (TA) to provide access control to TVM assigned memory and memory mapped I/O registers.

The T bit must be used only as defined for TDISP.

The DSM provides the following functions:

1. Authentication of device identities and measurement reporting.
2. Configuring the IDE encryption keys in the device.
3. Device interface management for locking TDI configuration, reporting TDI configurations, attaching, and detaching TDIs from TVMs.
4. Implementing access control and security mechanisms to isolate TVM provided data from entities not in the TCB of the TVM.

The TSM provides the following functions:

1. Provide interfaces to the VMM to assign memory, CPU, and TDI resources to TVMs.
2. Implements the security mechanisms and access controls (e.g., IOMMU translation tables, etc.) to protect confidentiality and integrity of the TVM data and execution state in the host from entities not in the TCB of the TVM.
3. Use TDISP protocol to manage the security state of the TDIs to be used by TVMs.
4. Establishing/managing IDE encryption keys for the host, and, if needed, scheduling key refreshes.

Secured messages as specified in Section [6.31] are used by TSM and DSM to communicate TDISP messages securely. The secure session establishment is used by the TSM to authenticate the DSM (Optionally, the DSM may be configured to authenticate the TSM, if required by the system design), negotiate cryptographic parameters, and establish shared keying material.

Once the SPDM secure session has been established, the session enters the application phase where all application data between the TSM and DSM are communicated using secured messages within the SPDM secure session. Two types of application data are used by TEE-I/O:

- IDE Key Programming – When IDE is required, the IDE_KM protocol is used for key programming. An IDE stream may also be established between two devices for peer-to-peer communication.
- TDI Management – the TSM uses the TDISP protocol to manage the TDI attach and detach to a TVM. The TSM steps the TDI through the TDISP states as part of the TDI lifecycle management process such as:
    - Locking a TDI configuration for assignment of the TDI to the TVM
    - Making the TDI operational if the TVM approves of the device
    - Detaching a previously assigned TDI from a TVM.

The DSM must track the SPDM session that was used to establish the IDE keys for an IDE stream. For the IDE stream to be usable for carrying TVM data, all the IDE keys for the IDE stream that will be used by the TDI assigned to a TVM must be programmed by the TSM.

Multiple TDIs (e.g., SR-IOV VFs) in a device may generate or receive transactions over the IDE stream established by the TSM and DSM to secure the communication links between the host and the device. One or more of these TDIs may be assigned to TVMs, and one of more of these TDIs may be assigned to legacy VMs. The TSM manages and tracks the TDISP state associated with the TDIs assigned to TVMs.

11.2. TDISP Rules

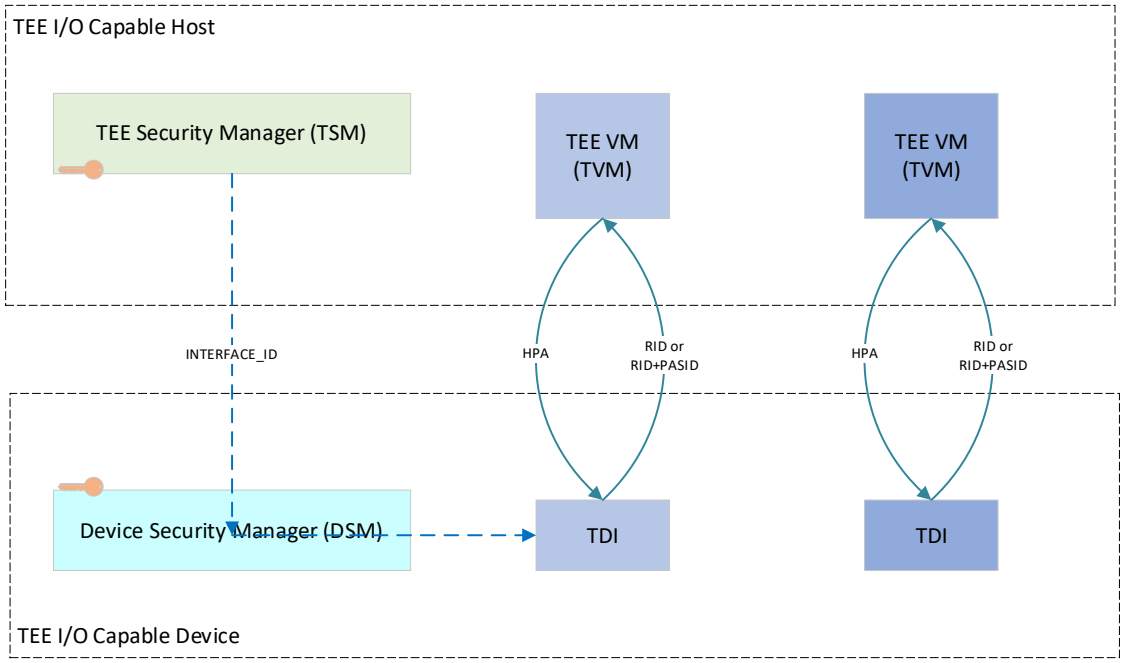


Figure 11-3 Identification of Requests

As illustrated in Figure 11-3, a TDI is managed by a specific DSM, and within the domain of that DSM it is necessary for each TDI to have a unique identifier, called an INTERFACE_ID, that is used in all TSM/DSM messages to indicate the applicable TDI. MMIO requests originated from the TVM are translated by the host and directed to appropriate TDI based on the HPA. Requests generated by the TDI contain the Requester ID (RID) of the function hosting the TDI, and may also have a PASID.

The INTERFACE_ID is composed of a FUNCTION_ID field that identifies the function of the device hosting the TDI and a Reserved field provided for future expansion (see Figure 11-4 TDI Identifier – INTERFACE_ID and Table 1 INTERFACE_ID Definition). Within the FUNCTION_ID, the Function Number and Device Number are assigned by the device/DSM. The Bus Number and Segment Number are assigned during system enumeration and must not be changed for a TDI in CONFIG_LOCKED and RUN (see below). If the Segment Number is known to the device, and Requester Segment Valid is Set, then the Requester Segment value must match the Segment Number for the device. The DSM must ensure that only valid TDIs are addressed.

Figure 11-4 TDI Identifier – INTERFACE_ID

Table 1 INTERFACE_ID Definition

Offset	Field	Size (Bytes)	Description
0	FUNCTION_ID	4	Identifies the function of the device hosting the TDI: 15:0 – Requester ID 23:16 – Requester Segment (Reserved if Requester Segment Valid is Clear) 24 – Requester Segment Valid 31:25 – Reserved
4	Reserved	8	Reserved

Each TDI in the device is associated with a TDISP state machine (see Figure 11-5).

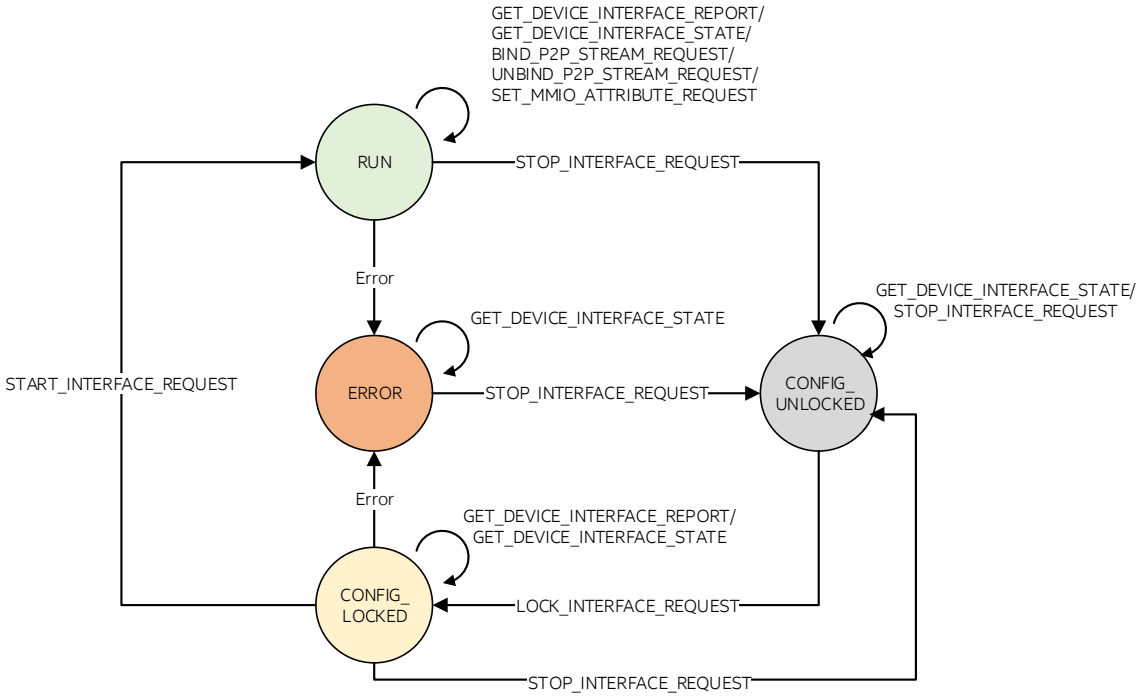


Figure 11-5 TDISP State Machine

The TSM steps the TDI through these security states as part of the TDI security lifecycle management process, such as locking a TDI configuration in preparation for assignment of the TDI to the TVM, transitioning the TDI to the operational state, and detaching the TDI from a TVM. A TDI is considered “locked” in CONFIG_LOCKED, and RUN. A TDI is considered “unlocked” when in ERROR and CONFIG_UNLOCKED.

For a TDI that supports assignment to Legacy VMs, if a TDI is assigned to a Legacy VM, the VMM assigns the TDI in CONFIG_UNLOCKED, and the TSM must ensure that the TDI remains in that state unless and until the TDI is removed from the Legacy VM and prepared for re-assignment to a TDI.

TDISP requires certain TLPs to be rejected, which means that first the TLP is processed according to the same rules that apply to all other TLPs, and only then are the TDISP-specific rules applied. If the result is a determination that the TLP must be rejected, the associated TDI must transition to ERROR where indicated, but no further error reporting or logging is required to be performed on that TLP, although it is optionally permitted on a case-by-case basis that a Request be handled as an Unsupported Request, and/or a Completion be handled as an Unexpected Completion, or that the TLP be dropped.

IDE Streams that are bound for use with TDISP are permitted to be used for non-TDISP TLPs as well, however:

- for such TLPs the T bit must be Clear in Transmitted TLPs, and must be ignored in Received TLPs (as defined in 6.33.4)
- the DSM must ensure that such TLPs are not allowed to access TVM confidential data.

Security properties for each state and transition rules are as follows:

- CONFIG_UNLOCKED
    - This is the default state. In CONFIG_UNLOCKED the VMM configures the TDI to be assigned to a TVM.
    - A TDI is not required to protect confidential data placed into it in this state; TVMs should not place confidential data into a TDI in this state.
    - Memory Requests originating within a TVM (as indicated by the T bit being Set) must be rejected in this state (see Section 11.2.1)
    - This state must be entered from any other state in response to the STOP_INTERFACE_REQUEST message. When the TDI transitions to CONFIG_UNLOCKED, the DSM must ensure all TVM confidential data held by the device in the context of that TDI cannot be exposed as plaintext outside the device and, to the maximum extent possible, the ciphertext associated with TVM data must not be exposed outside the device.
- CONFIG_LOCKED
    - Once the TDI configuration is finalized by the VMM, the VMM requests the TSM to lock the TDI configuration by transitioning the TDI to CONFIG_LOCKED.
    - The DSM must transition a TDI to CONFIG_LOCKED only in response to a LOCK_INTERFACE_REQUEST message.
    - Memory Requests originating within a TVM (as indicated by the T bit being Set) must be rejected in this state (see Section 11.2.1)
    - The LOCK_INTERFACE_REQUEST must indicate the Stream ID of the IDE stream to bind to the TDI, if IDE is required to secure the transfers to/from the device.
    - On entry to this state, the DSM must perform all necessary actions to lock the TDI configuration, and then must start tracking the TDI for changes that affect the configuration or the security of the TDI. Changes detected must be treated as an error, and the TDI transitioned to ERROR. An example list of architectural configurations registers that should be locked and tracked are specified in section 11.2.6 “Locked Device Configuration”. It is typically required for the DSM to track additional device-specific configurations, such as the configuration of work queues, device specific configurations such as MAC address, storage volume, etc.
    - The TVM may obtain the identity and measurements of the device hosting the TDI from the DSM, and also, if applicable, verify that an IDE stream has been established by the TSM between the host and the device. The TVM may request the TSM to obtain the TDI configurations using the GET_DEVICE_INTERFACE_REPORT request from the DSM.

The TVM may then evaluate the device identity and measurements, in addition to the TDI report to determine if the device meets the security requirements of the TVM.
    - If the TVM approves of the device, the TVM may request the TSM to transition the TDI to RUN.
- RUN
    - TDI resources are operational and permitted to be accessed and managed by the TVM.
    - On entry to this state, the DSM must continue tracking the TDI for changes that affect the configuration or the security of the TDI. Changes detected must be treated as an error, and the TDI transitioned to ERROR.
- ERROR
    - The TDI must not expose confidential TVM data.
    - Memory Requests originating within a TVM (as indicated by the T bit being Set) must be rejected in this state (see Section 11.2.1).
    - The TDI must restrict TLP operations as defined in Section 11.2.1 TLP Rules.
    - In ERROR, the TDI may still have confidential TVM data, and it is permitted that clearing this data be deferred until the receipt of a STOP_INTERFACE_REQUEST to transition the TDI to CONFIG_UNLOCKED.
        - It is permitted, but not required, that the TDI transition automatically from ERROR to CONFIG_UNLOCKED, if and only if the TDI first clears all TVM confidential data.

In CONFIG_LOCKED and RUN, the following conditions must be treated as errors, and cause the TDI to move to ERROR:

- Changes to TDI configuration that affect the configuration or the security of the TDI.
    - The Completion Status of Configuration Writes that modify TDI configuration must not be affected.
    - See also Section 11.2.6 DSM Tracking and Handling of Locked TDI Configurations (Informative)
- Changes to the Requester ID
- Resetting the TDI using a Function Level Reset. A PF reset affects all subordinate VF TDIs, whereas a VF reset affects only that TDI.
- Any IDE stream bound to the TDI transitions to the Insecure state.
- Receipt of a poisoned TLP or detecting data integrity errors in the device for data associated with that TDI, where the error is not recoverable.
- Other device specific conditions or changes in configuration that affect trust properties.

The STOP_INTERFACE_REQUEST message may be used to transition the TDI to CONFIG_UNLOCKED. When the TDI transitions back to CONFIG_UNLOCKED, it must ensure that all TVM confidential data held by the device cannot be exposed as plaintext outside the device. To the maximum extent possible the ciphertext associated with TVM confidential data must not be exposed outside the device.

The TSM is permitted to issue a GET_DEVICE_INTERFACE_REPORT in CONFIG_LOCKED and RUN.

The TSM is permitted to issue a GET_DEVICE_INTERFACE_STATE request in all states.

Some TDIs may support peer-to-peer communication with other devices. The Stream ID of the IDE stream(s) used for this communication are configured into the device using the BIND_P2P_STREAM_REQUEST message. This message must only be accepted by the DSM if the TDI is in RUN. The device must be ATS capable, and must have ATS enabled, to support peer-to-peer communication between TVM-assigned TDIs. The TSM and VMM must coordinate the use of ACS mechanisms to redirect device peer-to-peer traffic to the Root Complex, and the TSM must only issue a BIND_P2P_STREAM_REQUEST if the TLPs to be associated with that Selective IDE Stream will, in fact, travel between the two peer devices and not to/from the Root Complex.

Certain configurations require all requests originating from a TDI to be sent to the Root Complex, even if the device is in possession of a Translated Address that appears to refer to a resource within the TDI or

elsewhere within the device. Devices are strongly encouraged to implement redirection functionality to send all such requests to the Root Complex. A DSM must advertise the availability of such functionality so that the TSM can establish the correct configuration for the TDI and the rest of the system.

Some TDIs may support updating attributes of one or more MMIO ranges associated with the TDI using the SET_MMIO_ATTRIBUTE_REQUEST.

A TDI must not rely on I/O resources and I/O requests for providing functionality to a TVM. I/O resources must not be able to compromise the confidentiality or integrity of TVM data.

It is not required that Configuration Requests to a TDI be secured.

11.2.1. TLP Rules

This section defines the rules for TLPs that are associated with TVMs. In cases where it is possible to ensure communication is not tampered with, it may be possible to avoid the use of IDE, but some equivalent to constructs defined by IDE, particularly the T bit, may still be required, and how this is done is outside the scope of this specification. Under all circumstances, devices must ensure that device memory with the IS_NON_TEE_MEM attribute Clear can only be read/written within the context of an authorized TVM (indicated, when IDE is used, by the T bit being Set).

In all cases, traffic that has no security requirement is not required by TDISP to be secured by IDE or other means, although in specific platforms there may be additional security requirements, for example to apply IDE to all TLPs between the host and a particular device.

The following rules apply to the TDI acting as a Requester:

- In systems where IDE is required, a TDI in CONFIG_LOCKED or RUN must transmit TLPs with T bit Set only on an IDE stream bound to the TDI.
    - All TLPs not associated with a peer-to-peer IDE Stream must use the Default IDE Stream, if one has been configured.
- Memory Reads must only be issued while in RUN and must Set the T bit.
- For Memory Reads issued by the TDI while in RUN, the corresponding Completion(s) must be handled normally if and only if the TDI is still in RUN, and must otherwise be rejected.
    - A TDI in RUN must ignore the value of the T bit in Received Completions.
- Memory Writes other than MSI/MSI-X interrupts must only be issued while in RUN and must Set the T bit.

The MSI capability in the Configuration Space of the function hosting the TDI is not required to have a trusted configuration. With MSI-X, it is possible for the TVM to program the MSI-X table and MSI-X PBA in a trusted manner.

It is permitted for a TDI in CONFIG_LOCKED to issue an MSI/MSI-X only if the T bit is Clear. TDIs in RUN must observe the following rules of generating memory write requests to request service using MSI:

- MSI using address/data pair obtained from the MSI capability of the function must be generated with T bit Clear.
- MSI using address/data pair obtained from the MSI-X capability of the TDI must be generated with T bit Clear if the MSI-X table is not part of the MMIO ranges that are locked and reported in the DEVICE_INTERFACE_REPORT, else the MSI must be generated with T bit Set.

If a TEE-I/O capable device supports locking and reporting of MSI-X table, it must allocate the MSI-X table and PBA such that the entirety of the MSI-X table and PBA may be mapped onto separate isolated processor pages. The MSI-X Table and the PBA base addresses must each be aligned to the minimum processor page size on supported platforms. 64KB alignment is recommended to provide compatibility across various processor architectures.

TEE-I/O capable devices must locate the ST table, if supported, in the MSI-X table, if ST mode of operation is supported.

The following rules apply to the TDI acting as a Completer:

- Received Memory Requests targeting device memory with the IS_NON_TEE_MEM (see Section 11.3.22 SET_MMIO_ATTRIBUTE_REQUEST) attribute Clear must be handled normally if and only if all of the following conditions are satisfied:
    - the T bit for the Request is Set
    - the TDI is in RUN
    - if IDE is required, the Request is received on a Stream ID bound to the TDI;

  in all other cases the TLP must be rejected.
- Requests targeting device memory received with the T bit Set while in any state other than RUN must be rejected.
- The TDI's handling is not modified by TDISP state for Received Memory Requests targeting MMIO with IS_NON_TEE_MEM Set.
- The value of the T bit in the Completion(s) returned by the TDI must match the value of the T bit in the corresponding Request.

There are specific requirements for ATS Invalidation Requests, Invalidation Completions, Page Request Messages and PRG Responses, defined in Section 11.4.10.

VDMs not defined by PCI-SIG must be specified by the Vendor as TDI-specific or not TDI-specific.

A TDI is permitted to Transmit and to handle normally Received TDI-Specific Messages while in CONFIG_LOCKED, RUN, and ERROR if and only if the T bit is Set.

11.2.2. TDISP Message Transport

All TDISP messages must be transported between TSM and DSM using secured messages as specified by Secured CMA/SPDM (see Section 6.31.4 Secured CMA/SPDM) used within a secure session established between TSM and DSM as specified by [SPDM].

TDISP requires the TSM and the DSM to support AES-256-GCM as the Authenticated Encryption with Associated Data (AEAD) algorithm to protect data transferred using secured messages. The random data field of the secured messages must not be used for TDISP messages or IDE Key Management protocol messages, and this field must have a length of zero.

Function 0 of the device must support the DOE Extended Capability to establish the SPDM session and transport the secured messages.

The [SPDM] Requester role is assumed by the TSM in the host and the Responder role is assumed by the DSM. The TSM is permitted to use an untrusted channel (e.g., proxy through the VMM) to access the transport mechanism.

TDISP messages are transported as follows:

- The Requester (TSM) must use the [SPDM] VENDOR_DEFINED_REQUEST format
- The Responder (DSM) must use the [SPDM] VENDOR_DEFINED_RESPONSE format
- The StandardID field of VENDOR_DEFINED_REQUEST and VENDOR_DEFINED_RESPONSE message must contain the value assigned in [SPDM] to identify PCI-SIG.
- The VendorID field of VENDOR_DEFINED_REQUEST and VENDOR_DEFINED_RESPONSE message must contain the value assigned to identify PCI-SIG.
- The first byte of the VendorDefinedReqPayload/VendorDefinedRespPayload is the Protocol ID, and must contain the value 01h to indicate TDISP.

- The TDISP message forms the request or response payload in the VendorDefinedReqPayload or VendorDefinedRespPayload, respectively.
- The VENDOR_DEFINED_REQUEST/VENDOR_DEFINED_RESPONSE must in turn form the Application Data field of a Secured Message per [Secured SPDM].

The encapsulation is as illustrated in Figure 11-6:

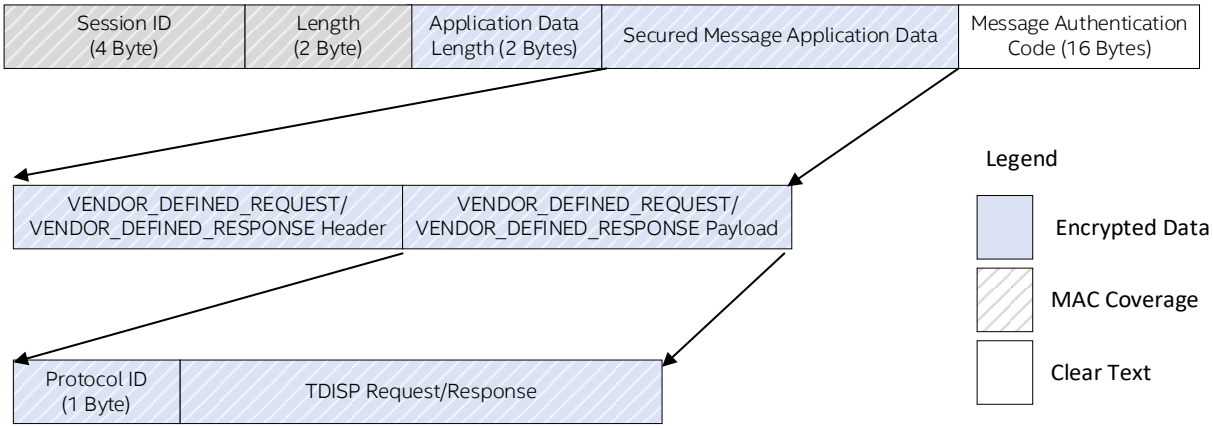


Figure 11-6 TDISP Request/Response Encapsulation

If a TDISP message is received that has not been transferred securely per [Secured SPDM], the received TDISP message must not be used, and must not result in a response.

11.2.3. Requirements for Requesters (TSM)

A Requester must not exceed the number of allowed outstanding requests to a specific DSM as indicated by NUM_REQ_ALL, and for a specific TDI as indicated by NUM_REQ_THIS (see Table 9 TDISP_CAPABILITIES). If the Requester has sent a request to a Responder and wants to send a subsequent request to the same Responder, then the Requester must wait to send the subsequent request until after the Requester completes one of the following actions:

1. Receives the response from the Responder for an outstanding request.
2. Times out waiting for a response.
3. Receives an indication, from the transport layer, that transmission of the request message failed.

A Requester is permitted to send simultaneous request messages to different Responders.

11.2.4. Requirements for Responders (DSM)

A Responder is not required to process more than NUM_REQ_THIS requests at a time. A Responder that is not ready to accept a new request message must either respond with a TDISP_ERROR response message with ERROR_CODE=BUSY or silently discard the request message.

If a Responder is working on a request message from a Requester, the Responder is permitted to respond with ERROR_CODE=BUSY.

If a Responder enables simultaneous communications with multiple Requesters, the Responder is expected to distinguish the Requesters by using the Session ID in the Secured Message.

11.2.5. Timing Requirements

TDISP inherits timing requirements from the SPDM protocol used to encapsulate the messages.

11.2.6. DSM Tracking and Handling of Locked TDI Configurations (Informative)

The DSM must track attempts to modify registers or other changeable device configuration controls affecting any Physical Function, Virtual Function, or non-IOV Function hosting a TDI in CONFIG_LOCKED or RUN. Precisely what must be tracked is implementation specific, and because the security properties of the device depend on correct implementation of these mechanisms, it is strongly recommended that persons skilled in building and validating secure hardware be deeply involved in the design and validation for all implementations.

Table 3 provides some general guidance regarding architecturally defined registers. Device-specific registers must be evaluated by the device vendors to determine if modifications to those are allowable. Device vendors must also evaluate additional device specific registers that are mapped into memory space or configuration space of the device to determine if they must be locked and tracked for modifications.

Read-only registers, hardware initialized registers, and registers used as selectors for reading out data (e.g., the Power Budgeting Data Select register) are excluded from this table. The DSM must ensure that attempts to modify those registers cannot affect the security of the TDI.

Table 2: Example DSM Tracking and Handling for Architected Registers

Register / Capability / Extended Capability	Example Response to Register Modification	Description
Cache Line Size **Latency Timer** **Interrupt Line**	Allowed	
Command	See description	Clearing any of the following bits causes the TDI hosted by the Function to transition to ERROR: • Memory Space Enable • Bus Master Enable Modification of other bits is allowed.
Status	Allowed	
BIST Register, **Base Address Registers,** **Expansion ROM Base Address**	Error	Transition hosted TDI to ERROR.
Power Management Capability	See description	If a power transition leads to the function losing its state, then the device transitions the TDI hosted by that function to ERROR.

Device Control, **Device Control 2,** **Device Control 3**	See description	Modifying state of any of the following bits causes the TDI hosted by the function to transition to ERROR: • Extended Tag Field Enable • Phantom Functions Enable • Initiate Function Level Reset • Enable No Snoop • 10-bit Tag Requester Enable • 14-bit Tag Requester Enable Modification of other bits is allowed.
Device Status, **Device Status 2,** **Device Status 3**	Allowed	
Link Control, **Link Control 2,** **Link Control 3,** **16.0 GT/s Control,** **32.0 GT/s Control,** **64.0 GT/s Control**	Allowed	If modifications to these registers lead to a Link Down condition, the IDE streams configured in the device transition to the Insecure state, and as a result TDIs bound to those streams transition to ERROR.
Link Status, **Link Status 2,** **16.0 GT/s Status,** **32.0 GT/s Status,** **64.0 GT/s Status**	Allowed	
MSI	Allowed	
MSI-X	See description	If MSI-X table was locked and reported, then any modifications cause transition to ERROR. Modifications are allowed otherwise.
Secondary PCIe, **Physical Layer 16.0 GT/s,** **Physical Layer 32.0 GT/s,** **Physical Layer 64.0 GT/s,** **Lane Margining at the Receiver,** **Flit Error Injection**	Allowed	If modifications to these registers lead to a Link Down condition, the IDE streams configured in the device transition to the Insecure state, and as a result TDIs bound to those streams transition to ERROR.
ACS, **Latency Tolerance Reporting**	Allowed	
L1 PM Substates	Allowed	If modifications to this register lead to a Link Down condition, the IDE streams configured in the device transition to the Insecure state, and as a result TDIs bound to those streams transition to ERROR.
Advanced Error Reporting	Allowed	As specified in Section 6.2.3.2.2, error mask register settings control reporting of detected errors, but do not block error detection.
Enhanced Allocation, **Resizable BAR,** **VF Resizable BAR,** **ARI,** **PASID**	Error	Transition the TDI to ERROR.
Virtual Channel, **Multi-Function Virtual Channel**	Allowed – see description	Device must enforce transaction ordering when TC/VC mapping is changed, or arbitration tables are updated.

Vendor Specific, Vendor Specific Extended, Designated Vendor Specific	See description	To be analyzed by the vendor based on the security principles provided by TDISP.
Multicast	Error	Enabling multicast mechanism is not supported for TEE-I/O capable devices. Transition hosted TDIs to ERROR.
Dynamic Power Allocation	Allowed	A device must guard against the part being placed outside of its specification. If the device cannot reliably operate within the power allocation through mechanisms like throttling, frequency control, etc. then the device transitions hosted TDIs to ERROR.
TPH Requester	See description	To be analyzed by the vendor based on the security principles provided by TDISP.
Precision Time Management, Hierarchy ID, Native PCIe Enclosure Management, Alternate Protocol	Allowed	
Protocol Multiplexing	Allowed	A TEE-I/O capable device that supports PMUX must not transmit or receive transactions for TDIs in CONFIG_LOCKED or RUN using PMUX packets. If modifications to this register lead to a Link Down condition, the IDE streams configured in the device transition to the Insecure state, and as a result TDIs bound to those streams transition to ERROR.
Shadow Functions	Not Applicable	Use of shadow functions is not permitted for TEE-I/O usages.
Data Object Exchange	Allowed	
Integrity and Data Encryption	See description	Modifying the stream control register, selective IDE RID association registers, or selective IDE address association registers of streams that are bound to locked TDIs is an error, and the device transitions the TDIs bound to such stream to ERROR.
Address Translation Services	Allowed	
Page Request, SR-IOV	Error	Transition hosted TDIs to ERROR.
VPD	Allowed	

11.2.7. TVM Acceptance of a TDI

A TVM must ask the following questions before it accepts a TDI into its TCB:

1. Is the identity of the device and the measurements reported by the device hosting the TDI acceptable?
2. Is there a SPDM secure session established between the TSM and the DSM, and does the identity authenticated by the TSM to setup the SPDM secure session match the identity reported to the TVM?
3. When IDE is required, were all IDE keys for the IDE stream used by the TDI established or verified by the TSM?

4. Has the VMM configured the TDI and mapped the TDI into the TVM address space as expected?

The TVM queries the TSM using a TSM-provided interface to determine the answers to questions 1, 2 and 3. The TVM then requests a report of the TDI configuration (see section 11.3.10 – GET_DEVICE_INTERFACE_REPORT) from the DSM and may use it, with support from the TSM to determine the answer to question 4.

If the answer to all of these questions is a yes, then the TVM may accept the TDI into its TCB.

11.3. TDISP Message Formats and Processing

11.3.1. TDISP Request Codes

The TDISP request codes table defines the TDISP request codes. All TDISP-compatible implementations must use the following TDISP request codes. Unsupported request codes must return a TDISP_ERROR response message with ERROR_CODE=UNSUPPORTED_REQUEST.

Table 3 TDISP Request Codes

Message	Code Value	Required/ Optional for Device	Legal TDISP states	Description
GET_TDISP_VERSION	81h	Required	N/A	This request message must retrieve a device's TDISP version
GET_TDISP_CAPABILITIES	82h	Required	N/A	Retrieve protocol capabilities of the device
LOCK_INTERFACE_ REQUEST	83h	Required	CONFIG_ UNLOCKED	Move TDI to CONFIG_LOCKED
GET_DEVICE_INTERFACE_ REPORT	84h	Required	CONFIG_ LOCKED, RUN	Obtain a TDI report
GET_DEVICE_INTERFACE_ STATE	85h	Required	CONFIG_ UNLOCKED, CONFIG_ LOCKED, RUN, ERROR	Obtain state of a TDI
START_INTERFACE_ REQUEST	86h	Required	CONFIG_ LOCKED	Start a TDI
STOP_INTERFACE_ REQUEST	87h	Required	CONFIG_ UNLOCKED, CONFIG_ LOCKED, RUN, ERROR	Stop and move TDI to CONFIG_UNLOCKED (if not already in CONFIG_UNLOCKED)

BIND_P2P_STREAM_ REQUEST	88h	Optional	RUN	Bind a P2P stream
UNBIND_P2P_STREAM_ REQUEST	89h	Optional	RUN	Unbind a P2P stream
SET_MMIO_ATTRIBUTE_ REQUEST	8Ah	Optional	RUN	Update attributes of specified MMIO range
VDM_REQUEST	8Bh	Optional	N/A	Vendor-defined message request

11.3.2. TDISP Response Codes

The Request Response Code field in the response message must specify the appropriate response code (see Table 4 TDISP Response Codes) for a request. On a successful completion of an operation, the specified response message must be returned. Upon an unsuccessful completion of an operation, the TDISP_ERROR response message must be returned. Undefined/unsupported response codes must be treated as if they were TDISP_ERROR.

Table 4 TDISP Response Codes

Message	Code Value (h)	Required/ Optional	Description
TDISP_VERSION	01	Required	Version supported by device.
TDISP_CAPABILITIES	02	Required	Protocol capabilities of the device.
LOCK_INTERFACE_RESPONSE	03	Required	Response to LOCK_INTERFACE_REQUEST
DEVICE_INTERFACE_REPORT	04	Required	Report for a TDI
DEVICE_INTERFACE_STATE	05	Required	Returns TDI state
START_INTERFACE_RESPONSE	06	Required	Response to request to move TDI to RUN
STOP_INTERFACE_RESPONSE	07	Required	Response to a STOP_INTERFACE_REQUEST
BIND_P2P_STREAM_RESPONSE	08	Optional	Response to bind P2P stream request
UNBIND_P2P_STREAM_RESPONSE	09	Optional	Response to unbind P2P stream request
SET_MMIO_ATTRIBUTE_RESPONSE	0A	Optional	Response to update MMIO range attributes
VDM_RESPONSE	0B	Optional	Vendor-defined message response
TDISP_ERROR	7F	Required	Error in handling a request

11.3.3. TDISP Message Format and Protocol Versioning

Table 5 TDISP Request Format defines the fields that are included in all TDISP messages. Unless otherwise specified, the following rules shall apply to all request and response messages in TDISP:

- Reserved, unspecified, or unassigned values in enumerations or other numeric ranges are reserved for future definition of the TDISP specification. Reserved numeric and bit fields must be written as zero (0) and ignored when read.
- Byte ordering of multi-byte numeric fields or multi-byte bit fields is "Little Endian" (that is, the lowest byte offset holds the least significant byte, and higher offsets hold the more significant bytes).
- All message fields, regardless of size or endianness, map the highest numeric bits to the highest numerically assigned byte in monotonically decreasing order until the least numerically assigned byte of that field.

All TDISP messages include the one Byte TDISPVersion field, which is divided into two sub-fields:

- Bits 7:4 - Major Version – The major version of the TDISP Specification. A device must not communicate by using an incompatible TDISP version value. The Major version is incremented when the protocol modification breaks backward compatibility.
- Bits 3:0 - Minor Version – The minor version of the TDISP specification. A specification with a given minor version extends a specification with a lower minor version if they share the major version. The Minor Version is incremented when the protocol modification maintains backward compatibility.

This version of TDISP is V1.0, represented as 10h.

See Section 11.3.5 “GET_TDISP_VERSION” for details on compatible version negotiation.

Table 5 TDISP Message Format

Offset	Field	Size (Bytes)	Description
0	TDISPVersion	1	The TDISPVersion field represents the version of the specification encoded as follows: • Bits 7:4 – Major Version • Bits 3:0 – Minor Version
1	MessageType	1	The code identifying the type of the message (see Table 3 TDISP Request Codes and Table 4 TDISP Response Codes).
2	Reserved	2	Reserved
4	INTERFACE_ID	12	The TDI’s ID, as defined in Section 11.2 TDISP Rules
16	TDISP message payload	Variable	Zero or more bytes that are specific to the MessageType

The device must fail the request and return the indicated response code if any of the error cases defined in Table 6 are detected:

Table 6 Generic Error Response Codes

Error Code	Description
INVALID_REQUEST	One or more of the fields of the request being invalid
VERSION_MISMATCH	Protocol version is unsupported or is not the agreed upon version
BUSY	Device cannot process the request due to being busy
UNSPECIFIED	Error due to an unspecified reason
VENDOR_SPECIFIC_ERROR	Error due to a vendor specific reason
INVALID_INTERFACE	The INTERFACE_ID indicated is not within the domain of the DSM

11.3.4. GET_TDISP_VERSION

This request message must retrieve the device's TDISP version. In all future TDISP versions, the TDISP_GET_VERSION and TDISP_VERSION response messages will be backward compatible with all previous versions. The Requester must begin the discovery process by sending a TDISP_GET_VERSION request message with major version 1h. All Responders must always support TDISP_GET_VERSION request message with major version 1h and provide a TDISP_VERSION response containing all supported versions, as the TDISP_GET_VERSION request message table describes. The Requester must consult the TDISP_VERSION response to select a common (typically highest) version supported. The Requester must use the selected version in all future communication of other requests. A Requester must not issue other requests until it has received a successful TDISP_VERSION response and has identified a common version supported by both sides. A Responder must not respond to TDISP_GET_VERSION request message with ERROR_CODE= RESPONSE_NOT_READY.

A host system is permitted to use GET_TDISP_VERSION to determine if a device support TDISP.

11.3.5. TDISP_VERSION

Table 7 TDISP_VERSION

Offset	Field	Size (Bytes)	Description
Payload (All fields in little endian format)			
16	VERSION_NUM_COUNT (N)	1	Number of version number entries. Minimum permitted value is 1.
17	VERSION_NUM_ENTRY[1-N]	1 x N	8-bit version entry where each entry is formatted as: • 7:4 – Major Version Number • 3:0 – Minor Version Number

11.3.6. GET_TDISP_CAPABILITIES

Used to retrieve the Responder's TDISP capabilities. TDISP protocol inherits the SPDM timing specifications, including the timing parameter CT used to determine the time the Responder must respond to a message needing cryptographic processing.

Table 8 GET_TDISP_CAPABILITIES

Offset	Field	Size (Bytes)	Description
Payload (All fields in little endian format)			

16	TSM_CAPS	4	TSM Capability Flags Bits 31:0 – Reserved

11.3.7. TDISP_CAPABILITIES

Capabilities supported by Responder.

Table 9 TDISP_CAPABILITIES

Offset	Field	Size (Bytes)	Description
Payload (All fields in little endian format)			
16	DSM_CAPS	4	DSM Capability Flags Bits 31:0 – Reserved
20	REQ_MSGS_SUPPORTED	16	Bitmask indicating each type of request message supported by the device, where the bit index corresponds to the message code defined in Table 3 minus 80h.
36	LOCK_INTERFACE_FLAGS_ SUPPORTED	2	Bitmask indicating lock interface flags supported by the device, where the bit index corresponds to the FLAGS field definition in Table 10. Lack of support for a specific flag indicates that software must not assume any particular device behavior regarding the related capability, unless it has device-specific knowledge through other means.
38	-	3	Reserved
41	DEV_ADDR_WIDTH	1	Device reports the number of address bits it supports. For example, a value of 52 in this field indicates support for Bits 51:0.
42	NUM_REQ_THIS	1	Number of outstanding Requests permitted by the DSM for this TDI.
43	NUM_REQ_ALL	1	Number of outstanding Requests permitted by the DSM for all TDIs managed by this DSM.

11.3.8. LOCK_INTERFACE_REQUEST

The LOCK_INTERFACE_REQUEST is used to move the TDI to CONFIG_LOCKED, provided that the DSM confirms that the device, including elements of Function 0 and the TDI itself, is acceptably configured and in an acceptable state.

The device must fail the request if any of the following errors are detected:

- Interface ID specified in the request is not hosted by the device.
- TDI is not in CONFIG_UNLOCKED.
- For TDIs where IDE is required:
    - The default Stream does not match the Stream ID indicated
    - The default stream does not have IDE keys programmed for all sub-streams
    - All IDE keys of the default stream were not configured over the SPDM session on which the LOCK_INTERFACE_REQUEST was received
    - Multiple IDE configuration registers in the device have been configured as the default stream
    - The default Stream is associated with a TC other than TC0
- Phantom Functions Enabled
- Device PF BARs configured with overlapping addresses
- Expansion ROM base address, if supported, configured to overlap with a BAR
- Resizable BAR control registers programmed with an unsupported BAR size
- VF BARs are configured with address overlapping another VF BAR, a PF BAR or Expansion ROM BAR
- Unsupported system page size is configured in the system page size register of SR-IOV capability
- Cache Line Size configured for LN requester capability (deprecated in PCIe Revision 6.0), if supported and enabled, does not match the system cache line size specified in the LOCK_INTERFACE_REQUEST or is configured to a value not supported by the device.
- ST mode selected in TPH Requester Extended Capability, if supported and enabled, does not correspond to a mode supported by the function hosting the TDI.
- Other device determined errors in the device or TDI configurations

The LOCK_INTERFACE_REQUEST binds and configures the following parameters into the TDI:

- MMIO_REPORTING_OFFSET – To avoid leaking physical addresses to a TVM, the TSM specifies a MMIO_REPORTING_OFFSET to be applied to all future DEVICE_INTERFACE_REPORT generated by this TDI. MMIO_REPORTING_OFFSET is a signed (2's complement) 64-bit value that must be added to all MMIO physical addresses reported by the TDI. In order to maintain host-specific page alignment, the TSM is permitted to supply the corresponding lower bits of MMIO_REPORTING_OFFSET as zero. The TSM must supply an offset that does not result in overflow/underflow.
- NO_FW_UPDATE when Set indicates that when this TDI is in CONFIG_LOCKED or RUN, the device must not accept firmware updates. This option allows certain TVM to opt-out of further firmware updates to the device once the TVM starts using the TDI. To perform a firmware update, a VMM must detach from the TEEs all TDI that are locked with NO_FW_UPDATE=1 from the TVM and move them to CONFIG_UNLOCKED.
- System cache line size.
- Whether MSI-X table and PBA in the function hosting the TDI must be locked. A TDI is permitted to lock the MSI-X table and PBA even if not directed to do so. If the MSI-X table and PBA are not locked, then they must not be reported in the DEVICE_INTERFACE_REPORT. If the MSI-X table and PBA are locked, transactions to access the MSI-X table and PBA without without the T bit Set must be rejected.
- BIND_P2P when Set indicates whether the Direct-P2P (device/device peer-to-peer) support may be enabled later via BIND_P2P_STREAM_REQUEST messages and a valid P2P address mask is specified in the request.
- ALL_REQUEST_REDIRECT when Set indicates that TDI must redirect all ATS Translated Requests upstream to the Root Complex, using the default Selective IDE Stream if one is

configured, to perform access checks. This includes TDI accesses to the local resources within TDI or other resources of a device that are based on a Translated Address.

On successful processing of the request, the device responds with a LOCK_INTERFACE_RESPONSE message.

Table 10 LOCK_INTERFACE_REQUEST

Offset	Field	Size (Bytes)	Description
Payload (All fields in little endian format)			
16	FLAGS	2	Bit 0 – NO_FW_UPDATE – When 1, indicates that device firmware updates are not permitted while in CONFIG_LOCKED or RUN. When 0, indicates that firmware updates are allowed while in these states. Bit 1 – System Cache Line Size – when 0 indicates the system CLS is 64 bytes and when Set indicates system CLS is 128 bytes. Bit 2 – LOCK_MSIX – Lock MSI-X table and PBA. Bit 3 – BIND_P2P – When 1, indicates that Direct-P2P support may be enabled later via BIND_P2P_STREAM_REQUEST messages and a valid P2P address mask is specified in this request. When 0, indicates that Direct-P2P is not allowed or enabled for this TDI. Bit 4 – ALL_REQUEST_REDIRECT – The TDI must redirect all ATS Translated Requests upstream to the Root Complex to perform access checks. Bits 15:5 – Reserved
18	Stream ID for Default Stream	1	Indicates the Stream ID for the Stream configured as the IDE default stream.
19	-	1	Reserved
20	MMIO_REPORTING_OFFSET	8	MMIO ranges reported in all DEVICE_INTERFACE_REPORT is reported with this offset added to the physical address
28	BIND_P2P_ADDRESS_MASK	8	Mask to be applied to target addresses for peer-to-peer transaction issued by the TDI using the BIND_P2P_STREAM_REQUEST stream (to clear-out any metadata information embedded in the address). This mask must be applied prior to using the Selective IDE Address Association mechanism. This mask is not applicable or applied for Requests bound to the Root Complex.

11.3.9. LOCK_INTERFACE_RESPONSE

LOCK_INTERFACE_RESPONSE is provided on successful handling of the LOCK_INTERFACE_REQUEST and the device having moved the TDI to CONFIG_LOCKED. The response message also provides a START_INTERFACE_NONCE that is generated when the TDI is locked. This nonce should be generated by the device in response to moving the TDI to CONFIG_LOCKED. This nonce must be destroyed when the TDI moves to CONFIG_UNLOCKED or ERROR from CONFIG_LOCKED. See Section 11.3.14 for additional rules regarding this nonce.

Generating a LOCK_INTERFACE_RESPONSE implies that the device has successfully completed the following operations:

- All in-flight and accepted work for the TDI, before lock request was received, are aborted
- All DMA for the TDI, before the lock request was received, are completed, or aborted
- If function hosting the TDI is capable of Address Translation Service (ATS), all ATS requests for the TDI, generated before the lock request was received, have completed, or aborted. The device must invalidate translations cached in the ATC by the Requester ID of the function hosting the TDI.
- If function hosting the TDI is capable of Page Request Interface Service (PRI), page requests for the TDI, generated before the lock request was received, have received responses or the TDI will discard page responses for outstanding page requests.
- Additional private resources that need to be assigned to the TDI by the DSM at the time of locking the TDI have been successfully allocated and assigned.
- DSM has carried out necessary actions on the device side to lock the TDI configuration and IDE configuration registers for the default stream. DSM has enabled mechanisms to track changes to the configurations of the TDI and the IDE configuration register for the default stream.

It is permitted for a TDI to return INVALID_DEVICE_CONFIGURATION in response to a LOCK_INTERFACE_REQUEST for implementation-specific reasons.

Table 11 LOCK_INTERFACE_RESPONSE

Offset	Field	Size (Bytes)	Description
Payload (All fields in little endian format)			
16	START_INTERFACE_NONCE	32	Device generated nonce to include in START_INTERFACE_REQUEST message.

The following table defines the error codes and conditions:

Table 12 LOCK_INTERFACE_REQUEST Error Response Codes

Error Code	Description
INVALID_REQUEST	Device supports IDE capability and - Keys have not been configured for all sub-streams of the default stream - Keys for the default stream were not configured using the SPDM session on which the LOCK_INTERFACE_REQUEST was received
INSUFFICIENT_ENTROPY	The device fails to generate nonce.
INVALID_INTERFACE_STATE	If the TDI is not in CONFIG_UNLOCKED.
INVALID_DEVICE_ CONFIGURATION	Locking the TDI failed due to invalid/unsupported device configurations.

11.3.10. GET_DEVICE_INTERFACE_REPORT

The GET_DEVICE_INTERFACE_REPORT is used to request a DEVICE_INTERFACE_REPORT from the device. The DEVICE_INTERFACE_REPORT may, in some cases, be larger than the requester can consume in a single response, so the requester is provided with the means to request a specific portion of the overall DEVICE_INTERFACE_REPORT to be sent with a given response.

The device must fail the request if any of the following errors are detected:

- Interface ID in the request is not hosted by the device
- TDI is not in CONFIG_LOCKED or RUN
- Invalid offset specified

Table 13 GET_DEVICE_INTERFACE_REPORT

Offset	Field	Size (Bytes)	Description
Payload (All fields in little endian format)			
16	OFFSET	2	Offset in bytes from the start of the report to where the read request message begins. The responder must send its report starting from this offset. For first GET_DEVICE_INTERFACE_REPORT request, the Requester must set this field to 0. For non-first requests, Offset is the sum of PORTION_LENGTH values in all previous DEVICE_INTERFACE_REPORT responses.

18	LENGTH	2	Length of report, in bytes, to be returned in the corresponding response. Length is an unsigned 16-bit integer. This value is the smaller of the following values: • Capacity of requester's internal buffer for receiving Responder's report. • The REMAINDER_LENGTH of the preceding DEVICE_INTERFACE_REPORT response. For the first GET_DEVICE_INTERFACE_REPORT request, the requester must use the capacity of the requester's receiving buffer. If offset=0 and length=FFFFh, the requester is requesting the entire report. The Responder is permitted to provide less than the requested length if the Responder's buffer length is limited.

11.3.11. DEVICE_INTERFACE_REPORT

Table 14 DEVICE_INTERFACE_REPORT

Offset	Field	Size (Bytes)	Description
Payload (all fields in little endian format)			
16	PORTION_LENGTH	2	Number of bytes of this portion of TDI report. This must be less than or equal to LENGTH received as part of the request. For example, the Responder is permitted to set this field to a value less than LENGTH received as part of the request due limitations on the Responder's internal buffer.
18	REMAINDER_LENGTH	2	Number of bytes of the TDI report that have not been sent yet after the current response. For the last response, this field must be 0 as an indication to the Requester that the entire TDI report has been sent.
20	REPORT_BYTES	PORTION_ LENGTH	Requested contents of TDI report

The TDI report is structured as follows:

Table 15 TDI Report Structure

Offset	Field	**Size (Bytes)**	Description
0	INTERFACE_INFO	2	Bit 0 – When 1, indicates that device firmware updates are not permitted while in CONFIG_LOCKED or RUN. When 0, indicates that firmware updates are permitted while in these states Bit 1 – TDI generates DMA requests without PASID Bit 2 – TDI generates DMA requests with PASID Bit 3 – ATS supported and enabled for the TDI Bit 4 – PRS supported and enabled for the TDI Bit 15:5 – Reserved
2	-	2	Reserved for future use.
4	MSI_X_MESSAGE_ CONTROL	2	MSI-X capability message control register state. Must be Clear if a) capability is not supported or b) MSI-X table is not locked.
6	LNR_CONTROL	2	LNR control register from LN Requester Extended Capability. Must be Clear if LNR capability is not supported. LN is deprecated in PCIe Revision 6.0.
8	TPH_CONTROL	4	TPH Requester Control Register from the TPH Requester Extended Capability. Must be Clear if a) TPH capability is not support or b) MSI-X table is not locked.
12	MMIO_RANGE_COUNT (N)	4	Number of MMIO Ranges in report

16	MMIO_RANGE	N * 16	Each MMIO Range of the TDI is reported with the MMIO reporting offset added. Base and size in units of 4K pages - 8 bytes – First 4K page with offset added - 4 bytes - Number of 4K pages in this range - 4 bytes – Range Attributes o Bit 0 – MSI-X Table – if the range maps MSI-X table. This must be reported only if locked by the LOCK_INTERFACE_REQUEST. o Bit 1 – MSI-X PBA – if the range maps MSI-X PBA. This must be reported only if locked by the LOCK_INTERFACE_REQUEST. o Bit 2 – IS_NON_TEE_MEM – must be 1b if the range is non-TEE memory. For attribute updatable ranges (see below), this field must indicate attribute of the range when the TDI was locked. o Bit 3 – IS_MEM_ATTR_UPDATABLE – must be 1b if the attributes of this range is updatable using SET_MMIO_ATTRIBUTE_REQUEST . o Bits 15:4 – Reserved o Bits 31:16 – Range ID – a device specific identifier for the specified range. The range ID may be used to logically group one or more MMIO ranges into a larger range.
16 + N * 16	DEVICE_SPECIFIC_INFO_LEN (L)	4	Number of bytes of device specific information
16 + N * 16 + 4	DEVICE_SPECIFIC_INFO	L	Device specific information

A TDI may generate (a) all DMA requests without PASID, (b) all DMA requests with PASID, or (c) some DMA requests with and others without PASID.

The Range ID is used to logically group the ranges reported in the report into logical groups.

MMIO ranges assigned via BAR(s) must be reported in ascending order starting with the lowest numbered BAR such that the first range corresponds to the first BAR and so on. The range ID reports the BAR equivalent Indicator (BEI). Values 0-7 of the Range ID are reserved to indicate the BEI. The device must report the BAR equivalent Indicator (BEI) for ranges associated with a PCIe BAR.

When reporting the MMIO range for a TDI, the MMIO ranges must be reported in the logical order in which the TDI MMIO range is configured such that the first range reported corresponds to first range of pages in the TDI and so on.

The device is permitted to include additional device specific information to the TVM in the report. The device specific information may be used to report configurations of the TDI and/or to enumerate capabilities of the TDI. Example of such device specific information include:

- A network device may include receive-side scaling (RSS) related information such as the RSS hash and mappings to the virtual station interface (VSI) queues, etc.
- A NVMe device may include information about the associated name spaces, mapping of name space to command queue-pair mappings, etc.
- Accelerators may report capabilities such as algorithms supported, queue depths, etc.

The following sequence diagram shows the high-level request-response message flow for Responder response when it cannot return the entire data requested by the Requester in the first response.

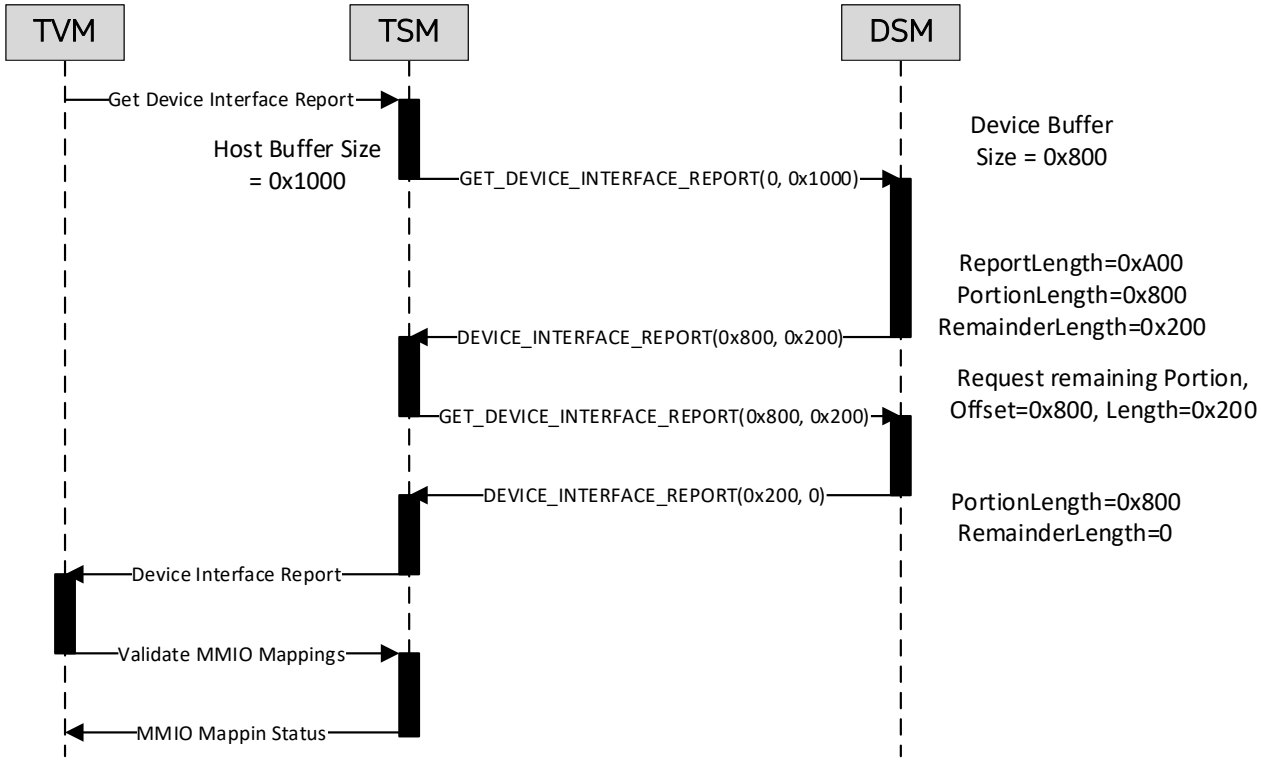


Figure 11-7 Example Flow Where DSM is Unable to Return Full Length Report

The following table defines the error codes and conditions:

Table 16 GET_DEVICE_INTERFACE_REPORT Error Response Codes

Error Code	Description
INVALID_REQUEST	OFFSET is invalid

INVALID_INTERFACE_STATE	The TDI is not in CONFIG_LOCKED.

11.3.12. GET_DEVICE_INTERFACE_STATE

The GET_DEVICE_INTERFACE_STATE is used to request a DEVICE_INTERFACE_STATE from the device.

The device must fail the request if the following error is detected:

- Interface ID in the request is not hosted by the device.

11.3.13. DEVICE_INTERFACE_STATE

Table 17 DEVICE_INTERFACE_STATE

Offset	Field	Size (Bytes)	Description
Payload (All fields in little endian format)			
16	TDI_STATE	1	TDI status 0 – CONFIG_UNLOCKED 1 – CONFIG_LOCKED 2 – RUN 3 – ERROR All others values Reserved

11.3.14. START_INTERFACE_REQUEST

The START_INTERFACE_REQUEST carries the interface ID of the TDI. This request is used to transition the TDI to RUN, where is managed and operated by the TVM. This request is expected to be generated by the TSM on request from the TVM.

The device must fail the request if any of the following errors are detected:

- If the interface ID in the request is not hosted by the device.
- START_INTERFACE_NONCE in the request is not valid i.e., does not match the nonce generated by the device in the LOCK_INTERFACE_RESPONSE.
- TDI is not in CONFIG_LOCKED.

If no errors are encountered, the device prepares to transition the referenced TDI to RUN. Moving the TDI to RUN may involve device side actions like enabling device side memory encryption, etc. The TDI must also invalidate the START_INTERFACE_NONCE before moving the TDI to RUN such that this nonce cannot be used again.

Table 18 START_INTERFACE_REQUEST

Offset	Field	Size (Bytes)	Description
Payload (All fields in little endian format)			
16	START_INTERFACE_NONCE	32	Device generated nonce for message (from LOCK_INTERFACE_RESPONSE)

11.3.15. START_INTERFACE_RESPONSE

The following table defines the error codes and conditions:

Table 19 START_INTERFACE_REQUEST Error Response Codes

Error Code	Description
INVALID_NONCE	START_INTERFACE_NONCE mismatch.
INVALID_INTERFACE_STATE	The TDI is not in CONFIG_LOCKED.

11.3.16. STOP_INTERFACE_REQUEST

The STOP_INTERFACE_REQUEST carries the interface ID of the TDI.

The device must fail the request the following error is detected:

- If the interface ID in the request is not hosted by the device.

In response to the STOP_INTERFACE_REQUEST the following actions must be performed:

- Abort all in-flight and accepted operations that are being performed by the TDI
- Wait for outstanding responses for the aborted operations
- All DMA read and write operations by the TDI are aborted or completed
- All interrupts from the TDI have been generated
- If function hosting the TDI is capable of Address Translation Service (ATS), all ATS requests by the TDI have completed or aborted. All translations cached in the device for ATS requests generated by this TDI have been invalidated.
- If function hosting the TDI is capable of Page Request Interface Service (PRI), no more page requests will be generated by the TDI. Additionally, either page responses have been received for all page requests generated by the TDI or the TDI will discard page responses for outstanding page requests.
- Scrub internal state of the device to remove secrets associated with the TDI such that those secrets will not be accessible.
- Reclaim and scrub private resources (e.g., memory encryption keys for device attached memories, etc.) assigned to the TDI.

The Device must generate the STOP_INTERFACE_RESPONSE once these actions are completed.

11.3.17. STOP_INTERFACE_RESPONSE

No request-specific responses are defined.

11.3.18. BIND_P2P_STREAM_REQUEST

A TDI is permitted to support peer-to-peer transactions secured from end-to-end between two devices. Such devices must support configuring one or more selective IDE Stream(s) such that the selective IDE stream configuration registers provide the address and Requester ID ranges for the peer device. Such peer-to-peer IDE streams must be used by a device only if the device supports Address Translation Services and the capability is enabled for the device.

The BIND_P2P_STREAM_REQUEST binds such peer-to-peer stream IDs to the TDI. The device must fail the request if any of the following apply:

- Interface ID in the request is not hosted by the device
- TDI does not support binding peer-to-peer streams
- TDI is not in RUN
- Stream ID specified does not have IDE keys programmed for all sub streams
- All IDE keys of the stream identified by the Stream ID were not configured over the SPDM session on which the LOCK_INTERFACE_REQUEST was received
- Multiple IDE configuration registers have been programmed with the same stream ID
- IDE configuration register for this stream is configured as the default stream
- Address and/or RID association registers of this streams IDE configuration registers overlap with other IDE configuration registers

In response to the request, DSM carries out necessary actions on the device side to lock the IDE configurations for the specified stream ID and enables mechanisms to track changes to the IDE configuration registers.

Through a device-specific mechanism the DSM ensures correctness of transaction ordering (i.e., if transactions were previously routed through the default stream ID specified by the LOCK_INTERFACE_REQUEST then the TDI has implemented fences or other mechanism before it starts using the peer-to-peer stream ID specified by this request).

Following processing the request, the device generates the BIND_P2P_STREAM_RESPONSE.

When a TDI generates a transaction, if the transaction’s address or Requester ID as applicable matches an IDE configuration register and the stream ID configured is one of the P2P streams bound to the TDI, then the device uses that P2P stream for the transaction. If the transaction does not match one of the P2P IDE streams, then the transaction uses the default stream identified by the stream ID bound using the LOCK_INTERFACE_REQUEST.

All ATS requests and requests with addresses for which translations were not previously obtained from the Translation Agent in the Root Complex must use the default stream identified by the stream ID bound at time of locking the TDI.

Table 20 BIND_P2P_STREAM_REQUEST

Offset	Field	Size (Bytes)	Description
Payload			
16	P2P_STREAM_ID	1	ID of the P2P stream to bind to this TDI.

11.3.19. BIND_P2P_STREAM_RESPONSE

The following table defines the error codes and conditions:

Table 21 BIND_P2P_STREAM_REQUEST Error Response Codes

Error Code	Description
INVALID_REQUEST	TDI does not support binding P2P streams. P2P_STREAM_ID is invalid Keys have not been configured for all sub-streams of the P2P stream Keys for the stream identified by P2P_STREAM_ID were not configured by the SPDM session on which the LOCK_INTERFACE_REQUEST was received IDE registers with configurations for the P2P_STREAM_ID is marked as the default stream. Multiple IDE registers are configured with the P2P_STREAM_ID The IDE registers configured for this P2P_STREAM_ID have overlaps with other valid IDE registers.
INVALID_INTERFACE_STATE	If the TDI is not in RUN.

11.3.20. UNBIND_P2P_STREAM_REQUEST

The UNBIND_P2P_STREAM_REQUEST unbinds a previously bound peer-to-peer stream IDs from the TDI. The device must fail the request if any of the following apply:

- Interface ID in the request is not hosted by the device
- TDI does not support binding peer-to-peer streams
- TDI is not in RUN
- Stream ID specified was not previously bound to this TDI

Following processing the request, the device generates the UNBIND_P2P_STREAM_RESPONSE.

An UNBIND_P2P_STREAM_RESPONSE implies that the device has successfully completed the following operations:

- All DMA read and write operations by the TDI using the specified P2P stream are aborted or completed
- Remove locking made active on the IDE configuration registers for this stream such that the IDE register may be reprogrammed without affecting the security of the TDI

If the device supports continuing the peer-to-peer operations following the unbind then through a device-specific mechanism the device ensures correctness of transaction ordering (i.e., if transactions were previously routed through this p2p stream then the TDI has implemented fences or other mechanism before it starts using the default stream ID).

Table 22 UNBIND_P2P_STREAM_REQUEST

Offset	Field	Size (Bytes)	Description
Payload			
16	P2P_STREAM_ID	1	ID of the P2P stream to unbind from this TDI.

11.3.21. UNBIND_P2P_STREAM_RESPONSE

The following table defines the error codes and conditions:

Table 23 UNBIND_P2P_STREAM_REQUEST Error Response Codes

Error Code	Description
INVALID_REQUEST	TDI does not support binding P2P streams. P2P_STREAM_ID is invalid P2P_STREAM_ID was not previously bound to this TDI.
INVALID_INTERFACE_STATE	If the TDI is not in RUN.

11.3.22. SET_MMIO_ATTRIBUTE_REQUEST

The SET_MMIO_ATTRIBUTE_REQUEST enables a TVM to update attributes of one or more MMIO ranges reported in the DEVICE_INTERFACE_REPORT. The MMIO ranges in a TDI that support updateable attributes are device specific.

The device must fail the request if any of the following apply:

- Interface ID in the request is not hosted by the device
- TDI does not support updateable MMIO attributes
- TDI does not support updateable MMIO attributes for the requested MMIO range
- TDI does not support the specified attribute for the requested MMIO range
- TDI does not support the value specified for the attribute
- TDI is not in RUN
- The MMIO range specified in the request is not associated with TDI

Responding with a failure is not fatal to the TDI and does not lead to a change in the TDI state.

Following processing the request, the device generates the SET_MMIO_ATTRIBUTE_RESPONSE.

IS_NON_TEE_MEM attribute may be updated to 1 to allow sharing the requested MMIO range with an entity not in the TVM trust boundary. Following the successful update of the attribute to 1, the specified MMIO range may be accessed using requests with T bit set to 0 or 1, or using a non-IDE Request. While the processing of the request is outstanding, a device may continue to reject Requests with the T bit Clear that access the MMIO range being updated.

IS_NON_TEE_MEM attribute may be updated to 0 to disallow sharing the MMIO range with an entity not in the TVM trust boundary. Following the successful update of the attribute to 0, the specified MMIO range may only be accessed using Requests with T bit Set, and Requests with T bit Clear must be rejected. While the processing of the request is outstanding, a device is permitted to continue to allow accesses via Requests with the T bit Clear.

A SET_MMIO_ATTRIBUTE_RESPONSE implies that the device has successfully completed updating the attributes for the specified MMIO range and the updated attributes are in affect for all subsequent accesses to this MMIO range.

Table 24 SET_MMIO_ATTRIBUTE_REQUEST

Offset	Field	Size (Bytes)	Description
Payload (All fields in little endian format)			
16	MMIO_RANGE	16	Base and size of the MMIO range to update attributes. - 8 bytes – First 4K page with offset added - 4 bytes – Number of 4K pages in this range - 4 bytes – Range Attributes o Bits 2:0 – Reserved – must be zero. o Bit 2 – IS_NON_TEE_MEM – set to 1b if the range is non-TEE memory. o Bits 15:3 – Reserved o Bits 31:16 – Range ID – a device specific identifier for the specified range.

11.3.23. SET_MMIO_ATTRIBUTE_RESPONSE

The following table defines the error codes and conditions:

Table 25 SET_MMIO_ATTRIBUTE_REQUEST Error Response Codes

Error Code	Description
INVALID_REQUEST	TDI does not support updateable MMIO attributes TDI does not support updateable attributes for requested MMIO range TDI does not support specified attribute for requested MMIO range TDI does not support the value specified for the attribute The range specified in the request is not associated with TDI
INVALID_INTERFACE_STATE	If the TDI is not in RUN.

11.3.24. TDISP_ERROR

The TDISP_ERROR is permitted to be used by the device to complete any of the requests issued to the device.

Table 26 TDISP_ERROR

Offset	Field	Size (Bytes)	Description
Payload (All fields in little endian format)			
16	ERROR_CODE	4	Error Code
20	ERROR_DATA	4	Error Data
24	EXTENDED_ERROR_DATA	Variable	Extended Error Data.

Table 27 Error Code and Error Data

Error Code	Value (h)	Description	Error Data	Extended error data
Reserved	0000	Reserved	Reserved	None
INVALID_REQUEST	0001	One or more request field is invalid.	Reserved	None

BUSY	0003	The Responder received the request message and the Responder decided to ignore the request message, but the Responder may be able to process the request message if the request message is sent again in the future.	Reserved	None
INVALID_INTERFACE_STATE	0004	The Responder received the request while in the wrong state, or received an unexpected request. For example, the GET_DEVICE_INTERFACE_REPORT before LOCK_INTERFACE_REQUEST, or any command between multiple GET_DEVICE_INTERFACE_REPORT	Reserved	None
UNSPECIFIED	0005	Unspecified error occurred.	Reserved	None
UNSUPPORTED_REQUEST	0007	Request code is unsupported	Request code	None
VERSION_MISMATCH	0041	The version in not supported	Reserved	None
VENDOR_SPECIFIC_ERROR	00FF	Vendor defined	Length of extended error data	See required formatting of extended error data for vendor defined errors
INVALID_INTERFACE	0101	INTERFACE_ID does not exist.	Reserved	None
INVALID_NONCE	0102	The received nonce does not match the expected one.	Reserved	None
INSUFFICIENT_ENTROPY	0103	The Responder fails to generate nonce.	Reserved	None
INVALID_DEVICE_ CONFIGURATION	0104	Invalid/Unsupported device configurations.	Reserved	None

The following table defines the EXTENDED_ERROR_DATA format for vendor defined TDISP_ERROR response messages:

Table 28 EXTENDED_ERROR_DATA

Offset	Field	Size (Bytes)	Description

0	REGISTRY_ID	1	ID of the registry assigning the VENDOR_ID 00h – PCI-SIG assigned vendor ID 01h – CXL assigned vendor ID
1	VENDOR_ID_LEN	1	Length of VENDOR_ID field.
2	VENDOR_ID	VENDOR_ID_ LEN	VENDOR_ID as assigned by the registry identified by REGISTRY_ID
2 + VENDOR_ID_LEN	VENDOR_ERR_ DATA	Variable	Vendor defined error data.

11.3.25. VDM_REQUEST

The following table defines the VDM_REQUEST format:

Table 29 VDM_REQUEST

Offset	Field	Size (Bytes)	Description
0	REGISTRY_ID	1	ID of the registry assigning the VENDOR_ID 00h – PCI-SIG assigned vendor ID 01h – CXL assigned vendor ID
1	VENDOR_ID_LEN	1	Length of VENDOR_ID field.
2	VENDOR_ID	VENDOR_ID_ LEN	VENDOR_ID as assigned by the registry identified by REGISTRY_ID
2 + VENDOR_ID_LEN	VENDOR_ DATA	Variable	Vendor defined data.

11.3.26. VDM_RESPONSE

The following table defines the VDM_RESPONSE format:

Table 30 VDM_RESPONSE

Offset	Field	Size (Bytes)	Description

0	REGISTRY_ID	1	ID of the registry assigning the VENDOR_ID 00h – PCI-SIG assigned vendor ID 01h – CXL assigned vendor ID
1	VENDOR_ID_LEN	1	Length of VENDOR_ID field.
2	VENDOR_ID	VENDOR_ID_ LEN	VENDOR_ID as assigned by the registry identified by REGISTRY_ID
2 + VENDOR_ID_LEN	VENDOR_ DATA	Variable	Vendor defined data.

11.4. Device Security Requirements

11.4.1. Device Identity and Authentication

A TEE-I/O capable device must implement the [SPDM] as the device secure communication protocol with the host. The device must use SPDM protocol to report the device identity and support the authentication. The security property defined in SPDM specification must be satisfied.

The device is recommended to implement the Device Identifier Composition Engine (DICE) architecture specified by the Trusted Computing Group (TCG). In this case, a DICE certificate must be returned in SPDM protocol and used to provide device identity and support authentication.

11.4.2. Firmware and Configuration Measurements

A TEE-I/O capable device must implement [SPDM] to return device measurements to the TSM. The TEE-I/O device may report hash-based measurement(s), and/or secure version number (SVN) to the host.

The device is recommended to implement TCG DICE. In this case, the device hash-based measurement and/or SVN must also be included in the DICE TCB info structure. The information reported in SPDM Measurement response and DICE TCB info must be consistent.

The device is permitted to support mutable firmware update. In this case, the device is recommended to follow established industry firmware resilience guidelines, such as NIST SP 800-193, to ensure the new firmware provides equal or better security.

The device is permitted to support runtime update without reset. Such a capability must be reported via INTERFACE_INFO, and can be blocked via NO_FW_UPDATE. A runtime update image must be of an equal or higher SVN if active TDIs are to be maintained. The device must report SPDM MEAS_FRESH_CAP to indicate if the device has capability to report fresh measurements or old measurements computed during last device reset. If a DICE device supports runtime update, the security property defined in DICE specification must still be satisfied, such as DICE certificate creation. An attempt to lower the SVN must be rejected by the device if there are active TDIs in CONFIG_LOCKED or RUN. Alternately, if the device has the capability to secure and clean all TVM data in a trusted manner before such a downgrade, the device is permitted to transition the interfaces in CONFIG_LOCKED or RUN to ERROR, terminate IDE streams used for TEE-I/O, and terminate the SPDM session with TSM before a firmware downgrade.

Devices that do not support runtime update when there are TDIs in CONFIG_LOCKED or RUN must handle a forced update by terminating the IDE streams used for TEE-I/O, and the SPDM session between the TSM and DSM, and transitioning associated TDI(s) to ERROR.

11.4.3. Securing Interconnects

TEE-I/O capable devices must support Integrity and Data Encryption (IDE) to protect transactions on the interconnect between the device and the Root Complex. Devices must support selective IDE streams between the Root Complex and the device. Use of IDE to protect transactions may not be required for RCiEP.

Peer-to-peer links, between the devices must be protected to avoid loss of confidentiality and integrity. Peer-to-peer must use IDE to secure communications. Other kinds of interconnects must be protected using interconnect specific extensions such that they provide equivalent security to address the threat model outlined in IDE.

The symmetric stream encryption keys and IV of each IDE Sub-Stream are secret and compromising these breaks the security of the solution. The device must implement adequate security measures to prevent leakage of the encryption key at rest and in use. The stream encryption keys must not be revealed in plaintext form outside the device. The device must not allow modifications to the stream encryption keys or the IV through untrusted mechanisms.

TEE-I/O capable devices must transition interfaces CONFIG_LOCKED, RUN to ERROR when the IDE stream(s) bound to those interface transition to Insecure state. The device must implement suitable mechanisms to contain propagation of data past the transition to Insecure state.

Receipt of a Completion with UR/CA or Completion timeout (following recovery retries) for requests initiated by a device on an interface in CONFIG_LOCKED, RUN indicates occurrence of an uncorrectable error (e.g., IOMMU translation tables corrupted, etc.) in handling the non-posted request. The device must transition the interface for which UR/CA was received to ERROR to stop consumption and propagation of errors if the error is not recoverable.

11.4.4. Device Attached Memory

Certain devices implement device attached memory where such memory is used by logic in the device to host the TVM data. The device must ensure the confidentiality of the TVM data stored in such memory devices such that the TVM data is not revealed as plaintext outside the device or to entities not in the TVM TCB. To the maximum extent possible the ciphertext associated with the TVM data must not be exposed outside the device. The device may additionally provide integrity properties on the TVM data.

IMPLEMENTATION NOTE: Security of Device Attached Memory

Certain devices may implement memory encryption as a mechanism to provide confidentiality of TVM data stored into those memory devices. Such devices may additionally provide integrity properties on the memory content. The configurations of memory encryption as well as other configurations related to the device attached memory in such devices is managed by the DSM. The TSM relies on the DSM to secure such configurations. Securing of memory configurations, establishing memory encryption, and all related security checks must be performed no later than in response to the first LOCK_INTERFACE_REQUEST received by the DSM for a device interface hosted by the device. If the memory configurations are not acceptable to meet TVM security requirements, then device interfaces must not be transitioned to CONFIG_LOCKED.

11.4.5. TDI Security

TEE-I/O capable devices must support [Secure SPDM] to establish a secure communication session between the TSM and DSM. The devices must support the TDI state and the device interface management protocol in TDISP for managing the device security states as they are assigned to TVMs and detached from TVMs.

All sub-streams of the IDE stream bound to the TDI must be programmed over the same SPDM session used to lock the interface. Attempting to configure IDE keys into a sub-stream using different SPDM sessions is an error and must be rejected. IDE key refresh must be accepted only on the SPDM session that was used to establish the initial IDE keys.

TDIs in RUN state may transmit/receive transactions with peer TDIs over peer-to-peer selective IDE streams if the device model supports such communication. The peer-to-peer selective IDE streams may be used for communication if the IDE keys for such streams were also configured with the same SPDM session as that was used to transition the TDI to CONFIG_LOCKED.

When the SPDM session used to program an IDE stream key enters session termination phase, all IDE streams configured with keys over that SPDM session must transition to an Insecure state and all TDI that were transitioned to CONFIG_LOCKED over that SPDM session must transition to ERROR.

When an IDE stream transitions to Insecure state, all TDIs in CONFIG_LOCKED/RUN bound to that IDE stream must transition to ERROR.

TEE-I/O capable devices must ensure that confidentiality and integrity of configurations and data associated with a TVM assigned TDI. Devices must implement suitable mechanisms to prevent leakage and tamper of such configuration and data from other TDIs including from the physical function of the device.

Devices may consider the administration functions provided through the physical function or the administrative queue of the device as untrusted. Typically, such administrative interfaces managed by the VMM are trusted by the TDIs (such as virtual function or TDIs) by virtual machines that include the VMM in the trust boundary. As the TVM may not include the VMM in the trust boundary, the administrative actions performed by the VMM may need to be mediated by the DSM to ensure that the administrative actions do not compromise the confidentiality or integrity of the TVM data, link encryption keys, SPDM session keys, and other secrets that are essential to the security of the solution. Administrative functions (e.g., QOS configurations, etc.) that are benign and could only lead to denial of service if misused may be allowed when the TDI is in CONFIG_LOCKED, RUN state.

TEE-I/O capable device should be designed to avoid or minimize the need for host driver intervention for TDI specific configuration or control operations where such operations if carried out maliciously may lead to confidentiality or integrity of the TVM data being compromised. Certain operations that only affect the quality of service or performance characteristics of the TDI but otherwise are benign to the functional

correctness and security of the interface may be carried out by host driver on behalf of the TVM. The host driver may not be in the trust boundary of the TVM assigned TDIs and may not be trusted to provide software emulation of TVM operations.

Changing the device configurations such as reprogramming the physical function or virtual function BAR, changing configurations of the TDIs, etc. may be used to temporarily drop a transaction originated by the TVM or lead to exposure of TVM confidential data (e.g., redirecting it to registers of a different virtual function than the one to which it is bound, etc.). Such configurations elements may be in the configuration space of the physical function, configuration space of the virtual functions, MMIO registers mapped by the physical function or the virtual function BARs, etc. Such configurations changes should be considered hostile actions and the TDI must be transition from CONFIG_LOCKED/RUN to ERROR.

Errors or failures encountered in the DSM or in other parts of logic in the device where the failures are not recoverable, or lead to the DSM losing state of the TDI, must lead to the device transitioning to a secure failed state where the TDIs in CONFIG_LOCKED/RUN transition to ERROR.

11.4.6. Data Integrity Errors

Receipt of a poisoned TLP on an interface in RUN indicates occurrence of uncorrectable data integrity errors. Receipt of such a TLP must transition the interface from RUN to ERROR to prevent bad data consumption and propagation. The device should implement suitable protection schemes such as parity or ECC on its internal data buffers and caches to detect data integrity errors. If uncorrectable data integrity errors were detected, then the affected interfaces must transition from CONFIG_LOCKED/RUN to ERROR and poison signaled to the requester as appropriate. The device may provide a mechanism to report and log the occurrence of such errors. Devices must scrub and clear information in such logs and reporting registers (e.g., syndrome) that may reveal confidential data.

11.4.7. Debug Modes

Devices may support multiple debug modes or debug capabilities. Some of the debug capabilities may allow a software debugger to collect statistics, error logs, etc. Such debug capabilities must not affect the security of the device, and must not lead to a compromise of the confidentiality or integrity of the TVM data provided to the device.

Debug configuration may, for example, be reported in SPDM measurements, such as non-invasive debug mode or invasive debug mode. A DICE device may report operation flags in DICE TCB info, such as debug mode.

Other debug modes may allow the debugger to affect the security of the device by providing mechanisms, for example, to bypass signature verification, trace data flowing through the device buses, affect the measurement process itself, etc. The device may support and use debug-mode identity certificates to identify such debug modes being active. This enables the TVM (and remote verifiers) to determine if the TVM may provide secrets to the device. TEE-I/O capable devices may restrict authorization of such debug modes to an early window following cold reset (i.e., the debug authorization occurs before secrets have been provided to the device or secrets in the device itself have been unlocked). When the debug authorization window is active, the device must not participate in SPDM session setup. Alternately, a TEE-I/O capable device may allow debug authorization to always occur but in response to the debug authorization request transition the IDE stream states to Insecure, terminate the SPDM session with the TSM, and transition all TDIs in CONFIG_LOCKED, RUN state to ERROR state prior to enabling the debug interface. Such devices provide the assurance that secrets already provided to the device by the TVM are not accessible to the debugger.

11.4.8. Conventional Reset

A conventional reset (cold, warm, or hot) leads to the device changing all its Port registers and state machines to their initialization values, and the TDISP state of all TDIs transitions to CONFIG_UNLOCKED.

Device reset architecture must ensure that all TVM data, IDE keys, other encryption keys (e.g., P2P links, intra-device interconnects, etc.) and SPDM session keys are cleared such that they are not exposed in plaintext through any mechanism following exit from the reset.

Devices may authorize debug following a reset and if the stream encryption keys are not scrubbed and cleared, they may become accessible to the debugger using debug tools. In addition to the stream encryption keys, TVM data held in clear text in the device (e.g., in the device's coherent caches, register files, SRAMs, etc.) may become accessible to debuggers using debug tools. The TEE-I/O capable device must implement suitable mechanisms to scrub residual secrets from device internal structures before authorizing debug access following a reset.

Devices may lose power as part of the conventional reset and may not have state coming out of the reset to determine whether the device was in use by a TVM or not. A TEE-I/O capable device should assume that prior to a conventional reset there may have been a TDI associated with a TVM and thus implement suitable mechanisms to ensure that residual data and secrets are not leaked in plaintext following such a reset.

The device measurement registers must be reset to their default values as part of a reset that requires firmware reload. Some devices may reload firmware on a warm reset, whereas others may require a cold reset or a D3 state transition.

11.4.9. Function Level Reset

A function level reset of a VF or non-IOV function must affect the TDI hosted by that function. A function level reset of the PF must affect all subordinate VF TDIs. A function level reset must transition all affected TDIs from CONFIG_LOCKED, RUN state to ERROR state such that a STOP_INTERFACE_REQUEST request is required to clean up the TDI state and scrub TVM data/secrets prior to the transition of the affected TDIs to CONFIG_UNLOCKED state.

A functional level reset does not affect active SPDM sessions or IDE streams.

11.4.10. Address Translation Services (ATS) and Access Control Services (ACS)

TEE-I/O capable devices that support ATS and have ATS enabled must generate translation requests and page requests for a TDI in RUN state with T bit Set.

If device supports IDE, then these requests must be generated over the IDE stream bound to the TDI by the LOCK_INTERFACE_REQUEST.

ATS requests not sent using the default IDE stream, must not be assumed to accurately reflect the permissions of that TDI, and as such if the device is caching host-memory, must not allow sharing between TEE TDIs and non-TEE TDIs of that cached value.

ATS Translation Requests issued by the TDI while in RUN must have the T bit Set, and the Translation Completion(s) must be received with the T bit Set. Translation Completion(s) received with the T bit Clear must transition the TDI to ERROR.

ATS Translated Read or Write Requests must only be issued by the TDI while in RUN, and must Set the T bit. Completions for ATS Translated Read Requests issued by the TDI while in RUN are permitted to be be received with the T bit Set or Clear.

ATS Invalidation Request and Invalidation Completion Messages are permitted to use or not use IDE. If IDE is used, Invalidation Requests applying to translations for a TDI in the RUN state are permitted to have the T bit Set or Clear. If the Invalidation Request uses IDE, then the Invalidation Completion must use the same IDE Stream as the Invalidation Request, and must match the T bit value from the Invalidation Request.

Page Request Messages must only be issued while in RUN, and must Set the T bit. Page requests from multiple interfaces in RUN are permitted to be grouped into a Page Request Group. A PRG Response must use the same IDE Stream as the corresponding Page Request, and must have the T bit Set. A violation of this rule must result in the TDI transitioning to ERROR.

Untranslated P2P requests generated by a TDI in RUN state must be redirected upstream towards the Root Complex over the default IDE stream bound to the TDI when the interface was transitioned to CONFIG_LOCKED state.

When there are no P2P streams bound to the interface, all translated P2P requests generated by a TDI must be directed upstream towards the Root Complex over the default IDE stream bound to the interface when the TDI was transitioned to CONFIG_LOCKED state. When there are P2P streams bound to the TDI, translated P2P requests generated by a TDI may be sent over a P2P stream bound to the TDI using the BIND_P2P_STREAM_REQUEST.

TEE-I/O capable devices must enforce integrity of the Address Translation Cache (ATC) such that the translations provided by the Root Complex cannot be modified through untrusted accesses.

Requirements on the device's PASID/ATS capabilities:

- Execute Permission Supported must be Clear
- Global Invalidate Supported must be Clear

Devices must, in all Translation Completions, treat the G bit as zero.

The use of Access Control Services (ACS) mechanisms for redirection must be coordinated with the device configuration to ensure that the correct Selective IDE Stream will be used. Specifically:

- ACS Translation Blocking
- ACS P2P Request Redirect
- ACS P2P Completion Redirect
- ACS Upstream Forwarding
- ACS P2P Egress Control
- ACS Direct Translated P2P

11.5. Requirements Placed on Host Security due to TDI Requirements

11.5.1. Address Translation

The property of memory being either TEE memory or non-TEE memory, must, as observed by a TVM executing on the host, match the view of memory as observed by a TDI assigned to that TVM. The translation agent (TA) is permitted to use the T bit being 1 to identify requests originated by a TDI in RUN state. The TVM relies on the TSM and TA to translate requests from TVM assigned TDIs such that:

1. An untrusted VMM cannot compromise the integrity of translation of Guest Physical Address or Guest Virtual Address to a physical address.
2. The Translation Agent (TA) ensures that Untranslated Requests transmitted by a TDI are checked against the access permissions for that TDI.

3. An Address Translation Cache (ATC) in the TDI, if enabled, is consistent with the Address Translation and Protection Tables (ATPT) associated with the TVM and changes to the ATPT are observed by both the CPU TLB, IOMMU TLB, as well as the ATC.
4. Identifiers such as Requester ID (and PASID) used for ATPT lookup are verified for integrity such that attempts to forge these identifiers are prevented.
5. Identifiers such as Requester ID and ITAGs used to process invalidation completions are verified for integrity such that attempts to forge these identifiers are prevented.

11.5.2. MMIO Access Control

The TSM must provide the following access controls on TVM assigned MMIO resources:

1. Verify that all MMIO resources reported by the device through the DEVICE_INTERFACE_REPORT have been made accessible to the TVM and the order in which these resources are mapped into the TVM address space match the expected mapping order.
2. Restrict a TVM and TVM assigned TDIs to only access MMIO resources that have been assigned to that TVM.
3. TLPs with the T bit Set may be generated to MMIO resources of a TVM only by accesses originated by that TVM or by components in the TCB of that TVM such as other TDIs assigned to that TVM that are in RUN state.

11.5.3. DMA Access Control

The TSM must provide the following access controls to TVM assigned memory and MMIO resources:

1. DMA through untranslated or already-translated requests to TVM memory must be allowed only from TDIs accepted by a TVM in its TCB. Such DMA may be to the confidential memory of the TVM or non-confidential memory accessible to the TVM. When a TVM accepts a TDI in its TCB, it accepts the entirety of the device in its TCB. A device is in the TCB of all TVMs that accept TDIs of that device in their TCBs. The TVM that accepts a device into its TCB trusts the device to not spoof identifiers used for DMA access control such as the source Requester ID (and the PASID). A TVM that does not accept a TDI of a device must not have the device in its TCB and such devices must not have access to TVM memory or MMIO resources using either untranslated or already-translated requests.

11.5.4. Device Binding

The TVM uses the authentication and measurement reporting protocol specified by [SPDM] to determine if the identity and measurements reported by the device hosting the TDI are acceptable prior to admitting the device into its TCB. The TVM needs to further determine if the authenticated device is presently bound to the host using an IDE stream (if applicable) and a SPDM session established by the TSM.

The TSM must provide a trusted mechanism to determine if:

1. A SPDM session is active between the TSM and the DSM in the device authenticated by the TVM.
2. IDE keys for the IDE stream used by that TDI have been established by the TSM.

11.5.5. Securing Interconnects

The symmetric stream encryption keys and IV of each IDE Sub-Stream are secret and compromising these breaks the security of the solution. The host must implement adequate security measures to prevent leakage of the encryption key at rest and in use. The stream encryption keys and IV must not be revealed outside the TSM and the TSM TCB. The host must not allow modifications to the stream encryption keys or the IV through untrusted mechanisms. These protections apply to all host-specific

mechanisms that contribute to maintaining confidentiality and integrity of IDE streams, for example Key Set change mechanisms or key refresh timers.

The host must program a unique encryption key for each IDE sub-stream and must not re-use the encryption keys when the IDE sub-stream keys are refreshed.

11.5.6. Data Integrity Errors

The host must provide data containment mechanisms to prevent consumption and further propagation of data in a poisoned TLP by a TVM or components in the TVM TCB.

It is strongly recommended that the host implement suitable protection schemes such as parity or ECC on its internal data buffers and caches to detect data integrity errors. If uncorrectable data integrity errors were detected, then the host must poison the data to prevent consumption and propagation by TVM, TVM assigned TDIs, or other components in the TVM TCB. The host must scrub registers that log information about the error, such as the syndrome, that could reveal confidential data.

11.5.7. TSM Tracking and Handling of Locked Root Port Configurations (Informative)

The TSM must track attempts to modify registers or other changeable configuration controls affecting Root Ports connecting to devices in CONFIG_LOCKED or RUN. Precisely what must be tracked is implementation specific, and because the security properties of the device depend on correct implementation of these mechanisms, it is strongly recommended that persons skilled in building and validating secure hardware be deeply involved in the design and validation for all implementations. Table 30 provides some general guidance regarding architecturally defined registers. Root Complex implementation-specific registers must be evaluated to determine if modifications to those are allowable. Read-only registers, hardware initialized registers, and registers used as selectors for reading out data (e.g., the Power Budgeting Data Select register) are excluded from this table. The TSM must ensure that attempts to modify those registers cannot affect the security of associated TDI(s).

Table 31 Example TSM Tracking and Handling for Root Port Configurations

Register / Capability / Extended Capability	**Example Response to Register Modification**	**Description**
Cache Line Size Latency Timer Interrupt Line	Allowed	
Command	See description	Clearing following bits causes the hosted IDE streams to transition to Insecure state: • Memory Space Enable • Bus Master Enable Modification of other bits is allowed.
Status	Allowed	

BIST Register, Base Address Registers, Expansion ROM Base Address, Primary Bus Number, Secondary Bus Number, Subordinate Bus Number, Root Port Segment Number	Error	Transitions hosted IDE streams to Insecure State.
I/O base/I/O Limit	Allowed	
Secondary Status Register	Allowed	
Memory Base/Memory Limit	Error	Transitions hosted IDE streams to Insecure State.
Prefetchable Memory Base/Prefetchable Memory Limit	Error	Transitions hosted IDE streams to Insecure State.
I/O Base Upper 16 Bits/I/O Limit Upper 16 Bits	Allowed	
Bridge Control	Allowed	
Power Management Capability	Allowed	If a power transition leads to the port losing its state, then the IDE streams hosted by that port transition to Insecure state.
Device Control	See description	Modifying Extended Tag Field Enable must cause streams hosted by the port to transition to Insecure state
Device Status	Allowed	
Link Control	See description	Disabling or retraining the link leads to IDE streams configured in the Root Port to transition to Insecure state
Link Status, Link Status 2	Allowed	
Slot Status	Allowed	
Root Control	Allowed	
Root Status	Allowed	
Device Control 2	See description	Modifying state of 10-bit Tag Requester Enable causes streams hosted by the port to transition to Insecure state. Modification of other bits is allowed.
Device Control 3	See description	Modifying state of 14-bit Tag Requester Enable causes streams hosted by the port to transition to Insecure state. Modification of other bits is allowed.
Link Control 2, Link Control 3	See description	Modifications to these registers are allowed. If the modifications lead to a link failure, the IDE streams configured in the port transition to Insecure state.
MSI, MSI-X	Allowed	

Secondary PCIe, Physical Layer 16.0 GT/s, Physical Layer 32.0 GT/s	See description	Modifications to these registers are allowed. If the modifications lead to a link failure, the IDE streams configured in the port transition to Insecure state.
Lane Margining at the Receiver	See description	Modifications to these registers are allowed. If the modifications lead to a link failure, the IDE streams configured in the port transition to Insecure state.
ACS	Allowed	
Latency Tolerance Reporting	Allowed	
L1 PM Substates	See description	Modifications to these registers are allowed. If the modifications lead to a link failure, the IDE streams configured in the port transition to Insecure state.
Advanced Error Reporting	Allowed	As specified in Section 6.2.3.2.2, error mask register settings control reporting of detected errors, but do not block error detection.
Enhanced Allocation	Error	Transitions hosted IDE streams to Insecure State.
Resizable BAR	Error	Transitions hosted IDE streams to Insecure State.
FRS Queueing	Allowed	
Flattening Port Bridge	Error	Transitions hosted IDE streams to Insecure State.
Virtual Channel	Allowed – see description	Root Port enforces transaction ordering when TC/VC mapping is changed, or arbitration tables are updated.
Vendor Specific	See description	To be analyzed by the vendor based on the security principles provided by TDISP.
Vendor Specific Extended	See description	To be analyzed by the vendor based on the security principles provided by TDISP.
Designated Vendor Specific	See description	To be analyzed by the vendor based on the security principles provided by TDISP.
RCRB Header	Allowed	
Root Complex Internal Link Control	See description	Modifications to this register is allowed. If the modifications lead to a link failure, the IDE streams configured in the port transition to Insecure state.
Multicast	Error	Enabling multicast mechanism is not supported for TEE-I/O operation. Transitions hosted IDE streams to Insecure State.
Dynamic Power Allocation	Allowed	A Root Port must guard against the part being placed outside of its specification. If the Root Port cannot reliably operate within the power allocation through mechanisms like throttling, frequency control, etc. then the Root Port must transition hosted IDE streams to Insecure state.
TPH Requester	Allowed	
DPC	Allowed	
Precision Time Management	Allowed	
Native PCIe Enclosure Management	Allowed	
Alternate Protocol	Allowed	
System Firmware Intermediary (SFI)	Allowed	

Protocol Multiplexing	Allowed	A Root Port that supports PMUX must not transmit or receive IDE TLPs using PMUX packets. If the modifications lead to a link failure, the IDE streams configured in the port transition to Insecure state.
Data Object Exchange	Allowed	
Integrity and Data Encryption	See description	Modifying the stream control register, selective IDE RID association registers, or selective IDE address association registers of streams that are bound to TVM assigned TDIs is an error, and the stream transitions to the Insecure state.
Flit Error Injection	See description	Modifications to this register is allowed. If the modifications lead to a link failure, the IDE streams configured in the port transition to Insecure state.

11.5.8. IDE Extended Capability registers

The host must enforce the integrity of IDE Extended Capability registers of the Root Ports used to configure streams used for TEE-I/O in the Root Port. Following mechanisms among others may be used to provide such integrity protection:

1. Restrict write access to such registers to the TSM or components in TSM TCB.
2. Transition the corresponding IDE stream to Insecure state if attempted to be modified by entities other than the TSM or components in TSM TCB.

TSM may provide an interface for the VMM to manage the lifecycle of IDE streams used for TEE-I/O. The VMM may use this TSM provided interface to configure, enable, disable, and reclaim (including non-graceful shutdown/reclaim) stream control registers. The details of such interfaces that may be provided by the TSM to the VMM are outside the scope of TDISP.

11.6. Overview of Threat Model and Mitigations

This section provides a very brief overview. It is strongly recommended that thorough threat model analysis be conducted by competent security expert(s) for all implementations.

11.6.1. Interconnect Security

The interconnect used to attach the device to the host needs to be secure against threats from physical attacks on the links. Adversaries are expected to have the ability to use lab equipment, interposers, custom devices, Switch firmware modifications, Switch routing table modifications, debug hooks in the Switches and Retimers, etc. to capture the data, re-order the data, or drop data transiting the links.

The adversary is expected to have the ability to perform reordering of transactions that are not legally allowed by the interconnect protocol.

The adversary may attempt to reprogram the encryption keys and replay protection counters associated with the link protection schemes to violate the confidentiality or integrity of the transactions on the link or to replay transactions on the link.

The adversary is expected to have the ability to craft custom devices or exploit vulnerabilities in authentic devices to attempt to spoof the identities used by the interconnect protocol (e.g., RID, PASID) to bypass access controls based on these identities.

These threats are addressed by use of IDE to secure the TLPs that carry TVM data.

The integrity protection on the TLP headers helps detect tampering of the identities such as RID and PASID. The use of selective IDE streams enables detection of attempts by the requester to use a RID

outside the range defined by the RID association registers. The device and host secure the IDE keys and SPDM secure session keys from entities not in the TVM TCB. The device detects modifications to IDE configurations as errors and transitions all associated TDIs to ERROR state. If an SPDM session transitions to session termination phase, then all IDE streams that had keys established over that session transition to insecure state and all TDI that were transitioned to CONFIG_LOCKED state over that SPDM session transition to ERROR. The host may either prevent modifications to IDE configurations or treat them as errors and transition the IDE streams to Insecure state.

A TDI can be associated with an IDE stream only if the IDE stream was keyed using the same SPDM secure session as that used for the LOCK_INTERFACE_REQUEST. A TDI can be associated with a P2P IDE stream only if the P2P IDE stream was keyed using the same SPDM secure session as that used for the BIND_P2P_STREAM_REQUEST. An IDE stream transitioning to Insecure state moves the associated TDIs to ERROR.

The host should guard against the following additional threats, using implementation-specific mechanisms, for security of IDE and TEE-I/O:

- Configurations used by the host to route addresses to Root Ports should be protected to prevent rerouting of transactions to unintended destinations:
  - TVM initiated transactions to device memory
  - P2P requests routed through the host
  - Device initiated transactions to host memory
- Reset of the host Root Ports and other logic blocks to place them in HwInit state
- Debug modes that affect the confidentiality and integrity of IDE keys, IVs, routing configurations, and other functions of the host that affect TEE-I/O security objectives.

11.6.2. Identity and Measurement Reporting

The adversary is expected to have the ability to build custom devices that mimic a legitimate device but be otherwise maliciously crafted to compromise the confidentiality and/or integrity of the TVM confidential data provided to the device.

The adversary may have control of the version of device firmware that loads even in authentic devices and be able to exploit vulnerabilities that may exist in specific versions of the firmware.

The devices may support debug capabilities that the adversary can invoke to affect the confidentiality and/or integrity data in the device or functional operations of the device.

The adversary is expected to use physical accesses and/or access to local/remote debug interfaces among others to attempt to subvert the root of trust of the device including the services provided by the device root of trust for measurement, reporting, identity, authorization, or update.

The adversary may have controls that allow downgrading a device firmware after the TVM has verified the measurements and may use this ability to exploit vulnerabilities in the downgraded version.

The adversary may have control on the version of firmware/software components that are loaded in the host that are in the TVM trust boundary and be able to exploit vulnerabilities that may exist in specific versions of the firmware/software.

The adversary may have physical access to the host and/or access to local/remote debug interfaces among others to attempt to subvert the TVM root of trust in the host.

These threats are mitigated by use of [SPDM] for identity and measurement reporting.

The devices implement security mechanisms to protect the root of trust in the device. Devices implement secure mechanisms to provision the device root of trust. Devices implement secure root of trust for measurement and protect the integrity of the measurement registers. Devices protect the Root of Trust (RoT) and Root of Trust for Measurement (RTM) from debug modes. TSM provides device binding

information to the TVM such that the TVM can answer the questions outlined in Section 11.2.7 to determine if the device if it is accepting a device in a secure state into its TCB.

11.6.3. TDI Assignment and Detach

Assigning a TDI to a TVM includes providing access to the MMIO resources of the TDI, and establishing access controls on DMA from the TDI to the TVM memory.

The adversary may be software outside the trust boundary of the TVM, including other TVMs, devices that are not trusted by the TVM, or an adversary using debug interfaces, etc. to influence the correctness and/or integrity of the MMIO resource assignment to a TVM and correctness and/or integrity of the DMA translation tables.

An adversary such as an untrusted VMM, or a TDI controlled by software outside the TVM trust boundary (including an unrelated/untrusted TVM), may attempt to maliciously access the MMIO resources assigned to a TVM to affect the confidentiality and/or integrity of the registers and MMIO mapped resources in the device.

An adversary such as an untrusted VMM may attempt to map the MMIO resources assigned to a TVM in an incorrect order and thereby attempt to trick a TVM to access a wrong set of registers and/or resources in the TDI.

The adversary may attempt to exploit properties like address decode priorities by assigning overlapping MMIO resources to two or more TDIs to redirect the transactions.

The adversary may attempt to reprogram a TDI that is being used by a TVM to affect the functioning of the TDI to influence the confidentiality and/or integrity of the transactions on the link or confidential data in the TDI. Examples of such reprogramming may include modifying the MMIO BAR of the device to attempt to drop transactions.

The adversary may attempt to asynchronously change the device state by issuing resets to the device to cause the device to drop established protections when a TVM is actively using the device.

The adversary may have the ability to launch a maliciously crafted TVM that collaborates with the adversary in violating the confidentiality and integrity of the TVM data.

These threats are mitigated by the use of TDISP.

TDISP provides the protocol and security requirements to lock TDI configurations using a LOCK_INTERFACE_REQUEST, obtain a report of the locked TDIs using a GET_DEVICE_INTERFACE_REPORT, securely enabling the memory space and DMA for TVM access using START_INTERFACE_REQUEST. A nonce generated by the device when the TDI is transitioned to CONFIG_LOCKED and verified on request to transition to RUN provide the property that all transitions through the TDISP state machine occur due to TDISP requests generated in the same SPDM secure session.

The DEVICE_INTERFACE_REPORT provides a trusted report of the TDI configurations, the list of MMIO resources associated with the TDI, and the order in which they must be mapped into the TVM address space. The TVM and TSM use the DEVICE_INTERFACE_REPORT to enforce that all MMIO resources of a TDI are assigned to the TVM to which the TDI is assigned and that the MMIO resources are mapped into the TVM address space in the expected order. Section 11.5.1 specifies the requirements on the TSM to enforce address translation integrity. Section 11.5.2 specifies the requirements on the TSM to enforce MMIO access control. The TSM through these access control mechanisms ensures that accesses with T bit Set can be generated to a MMIO register only by the TVM that has been allocated those resources. The device in RUN must only allow a Request to access memory with IS_NON_TEE_MEM Clear when the Request's T bit is Set. If the device supports IDE capability, accesses targeting memory with IS_NON_TEE_MEM Clear must be accepted only on an IDE stream bound to the TDI.

TSM uses host specific mechanisms to enforce DMA access control. The TSM uses the STOP_INTERFACE_REQUEST to ensure that all worked queued into a TDI has been drained and stopped before the resources allocated to a TVM can be reclaimed.

The TSM is expected not to bind any P2P streams using BIND_P2P_STREAM_REQUEST message unless the host supports ATS translation, and the Root Complex performs the correct TEE access checks at the time an ATS Translation Request is issued. Upon receipt of a Translation Completion that resolves to an address described by a P2P stream, the device receiving the Translation Completion is assured that it has full permission to request access to the specified resource. Similarly, any device that has a P2P stream configured and that receives a request on that stream with the T bit Set is assured that the sender has full permission to request access to the specified resource. No additional access checks are required or expected to be performed by the receiver.

TSM allows a TVM to update MMIO attributes of a TDI using SET_MMIO_ATTRIBUTE_REQUEST if the pages in the MMIO range of the request were allocated to the TVM making the request.

Devices track configurations of TDIs in CONFIG_LOCKED to detect attempts to reconfigure the TDI. Function level resets transition the TDI to ERROR. Conventional resets require the device to clear residual TVM secrets, IDE secrets, and SPDM session secrets such that they are not accessible to entities outside TVM trust boundary.